



DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA,
ELETTRONICA E SISTEMISTICA

Corso di Laurea Robotics and Automation Engineering

Corso di Laboratorio di Meccatronica e Robotica Mobile

Sistema di navigazione autonoma UAV in 3D basato su ROS 2, Mapping e percezione visiva

**Prof.
Marco Lupia**

**Giuseppe Coppola: 263624
Domenico Bellocci: 264131**

ANNO ACCADEMICO 2025/26

Indice

Indice	1
1 Introduzione	3
2 Architettura del Sistema	4
3 Ambiente di Simulazione e Modello del Drone (x500)	6
3.1 L'Ambiente Operativo: Magazzino Industriale	6
3.2 Modello del Quadricottero x500	10
3.2.1 Catena Cinematica	10
3.2.2 Modello dei Motori	11
3.2.3 Suite Sensoriale	11
3.2.4 Plugin di simulazione	13
4 Progettazione e Implementazione del Sistema di Controllo del Volo	14
4.1 Architettura di Controllo	14
4.1.1 Formalizzazione Matematica	15
4.2 Anello Esterno	16
4.3 Anello Interno	17
4.4 Mixing Matrix e Attuazione	18
5 Costruzione della Mappa (RTAB-Map)	21
5.1 Architettura e Sincronizzazione Sensoriale	21
5.2 Pre-Processing e Generazione Voxel Grid	22
5.3 Gestione del Grafo SLAM e Policy di Aggiornamento	23
5.4 Topic ROS 2 e Visualizzazione	24
6 Pianificazione della Traiettoria e Navigazione Autonoma 3D	26
6.1 Architettura Generale	26
6.2 Discretizzazione Spaziale e Gestione della Voxel Grid	27

6.3	Weighted A^* 3D	28
6.3.1	Semplificazione Greedy	31
6.3.2	Filtro di Spaziatura Minima	32
6.3.3	Fusione Collineare	32
6.4	Generazione di Traiettorie e Parametrizzazione Temporale	32
6.4.1	Pre-filtraggio Spaziale e Assegnazione Temporale	33
6.4.2	Formulazione Analitica delle Spline	33
6.5	Sincronizzazione Temporale e Feedforward Cinematico	34
7	Navigazione Basata su Human Following	36
7.1	Architettura di Percezione Visiva	36
7.2	Compensazione Odometrica e Ray-Casting	37
7.3	Filtraggio tramite Filtro di Kalman	38
7.4	Logica di Inseguimento e Target Selection	39
8	Simulazione e Validazione Sperimentale	42
8.1	Decollo e Stabilizzazione in Hovering	42
8.2	Navigazione 3D Autonoma	44
8.2.1	Scenario A: Navigazione Planare a Quota Costante	44
8.2.2	Scenario B: Navigazione Tridimensionale a Quota Variabile	46
8.2.3	Analisi del Grafo Computazionale ROS 2	48
8.3	Navigazione con Humang Following	50
8.3.1	Analisi Spaziale della Traiettoria di Inseguimento	50
8.3.2	Analisi del Grafo Computazionale ROS 2	52

Capitolo 1

Introduzione

Il presente progetto descrive lo sviluppo di un sistema di navigazione autonoma tridimensionale per un drone quadricottero in ambiente simulato, realizzato mediante ROS 2 Humble e Gazebo.

L'obiettivo è realizzare un'architettura modulare in grado di consentire al drone di esplorare ambienti sconosciuti, costruire una mappa tridimensionale dell'ambiente, pianificare traiettorie evitando gli ostacoli ed eseguire autonomamente il volo verso posizioni assegnate.

Oltre alla navigazione autonoma, il sistema integra una funzionalità di **human following**, nella quale il drone rileva una persona attraverso una telecamera RGB, ne stima la posizione rispetto all'ambiente e genera automaticamente nuovi obiettivi di navigazione, consentendo un inseguimento autonomo mantenendo una distanza prefissata.

L'intero sistema è stato sviluppato secondo un'architettura modulare, suddividendo le funzionalità di simulazione, mapping, pianificazione, controllo e visione artificiale in pacchetti ROS 2 indipendenti, così da facilitarne lo sviluppo, il collaudo e l'estensione.

Nei capitoli successivi vengono descritte le principali componenti del sistema e il loro funzionamento.

Capitolo 2

Architettura del Sistema

Il progetto è stato sviluppato seguendo un'architettura modulare basata su ROS 2, nella quale ogni componente svolge un compito specifico e comunica con gli altri mediante topic.

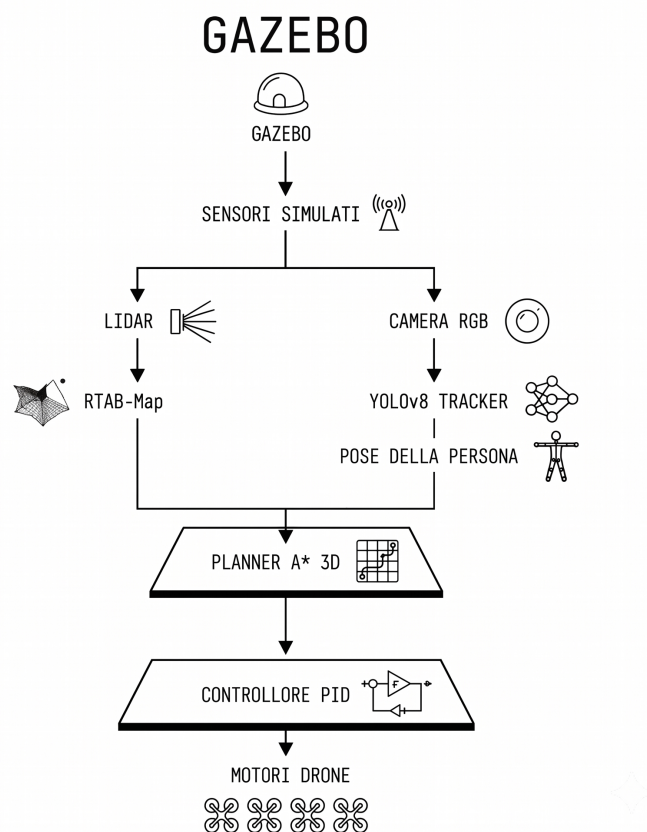


Figura 2.1: Diagramma del flusso dei dati e di controllo

Le principali funzionalità del sistema sono suddivise nei seguenti pacchetti:

- **drone_main**: contiene la descrizione del drone, i file URDF, l'ambiente di simulazione in Gazebo e i file di launch necessari per l'avvio del sistema.

- **drone_slam:** esegue solamente la costruzione della mappa tridimensionale mediante RTAB-Map, utilizzando la nuvola di punti generata dal LiDAR.
- **drone_navigation:** implementa il pianificatore tridimensionale basato sull'algoritmo A^* , che calcola una traiettoria priva di collisioni sfruttando la mappa prodotta dal modulo SLAM.
- **drone_control:** realizza il controllo del quadricottero mediante regolatori PID.
- **drone_follow:** implementa il sistema di rilevamento e inseguimento della persona mediante YOLOv8, generando automaticamente nuovi obiettivi di navigazione.

Capitolo 3

Ambiente di Simulazione e Modello del Drone (x500)

Questo capitolo descrive l'ambiente di simulazione basato su Gazebo e il modello fisico-cinematico del quadricottero x500. Vengono analizzati i parametri spaziali dello scenario logistico e l'integrazione della suite sensoriale a bordo del robot, elementi fondamentali per la validazione degli algoritmi di navigazione autonoma e visione artificiale.

3.1 L'Ambiente Operativo: Magazzino Industriale

L'ambiente di simulazione è definito dal file *warehouse.sdf* e riproduce un contesto logistico sfidante per la navigazione spaziale autonoma.

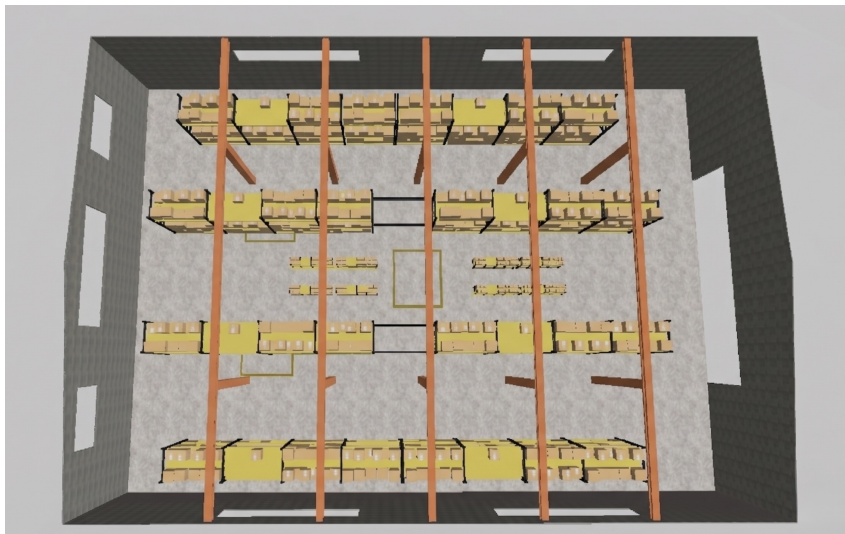


Figura 3.1: Ambiente di simulazione: magazzino industriale con scaffalature e ostacoli.

La topologia del magazzino è caratterizzata da corridoi stretti delineati da ostacoli statici, tra cui molteplici scaffalature industriali (*big_shelf* e *shelf*) e barre strutturali poste in

orizzontale (definiti come *beam*) che restringono gli spazi di manovra.

Per garantire la stabilità della simulazione e la corretta propagazione dei dati sensoriali ai nodi ROS, l'SDF dichiara una serie di plugin di sistema fondamentali:

```
<plugin filename="gz-sim-physics-system" name="gz::sim::systems::Physics"/>
<plugin filename="gz-sim-user-commands-system" name="gz::sim::systems::UserCommands"/>
<plugin filename="gz-sim-scene-broadcaster-system" name="gz::sim::systems::SceneBroadcaster"/>
<plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
  <render_engine>ogre</render_engine>
</plugin>
```

Sono inclusi il motore fisico e i sistemi di gestione dei comandi e della scena. Il modulo sensori abilita la simulazione dei dispositivi percepiti dal robot, con rendering basato su Ogre.

```
<physics type="ode">
  <max_step_size>0.004</max_step_size>
  <real_time_factor>1.0</real_time_factor>
  <real_time_update_rate>50</real_time_update_rate>
</physics>
```

Il modello fisico utilizza il motore Open Dynamics Engine (ODE). Il passo di simulazione è limitato da **max_step_size**, mentre **real_time_factor** e **real_time_update_rate** regolano rispettivamente la velocità rispetto al tempo reale e la frequenza di aggiornamento della simulazione.

```
<gravity>0 0 -9.8</gravity>
<magnetic_field>6e-06 2.3e-05 -4.2e-05</magnetic_field>
<atmosphere type="adiabatic"/>
<scene>
  <grid>false</grid>
  <ambient>0.4 0.4 0.4 1</ambient>
  <background>0.7 0.7 0.7 1</background>
  <shadows>false</shadows>
</scene>
```

Il sistema simula condizioni ambientali realistiche impostando gravità e campo magnetico terrestri e un'atmosfera adiabatica. La scena è semplificata disattivando griglia e ombre e definendo colori ambientali uniformi per una visualizzazione più chiara.

```

<light name="sunUTC" type="directional">
  <pose>0 0 500 0 -0 0</pose>
  <cast_shadows>false</cast_shadows>
  <intensity>1</intensity>
  <direction>0.001 0.625 -0.78</direction>
  <diffuse>0.904 0.904 0.904 1</diffuse>
  <specular>0.271 0.271 0.271 1</specular>
  <attenuation>
    <range>2000</range>
    <linear>0</linear>
    <constant>1</constant>
    <quadratic>0</quadratic>
  </attenuation>
  <spot>
    <inner_angle>0</inner_angle>
    <outer_angle>0</outer_angle>
    <falloff>0</falloff>
  </spot>
</light>

```

La scena utilizza una luce direzionale che simula il sole, con intensità unitaria e direzione definita per l'illuminazione globale. Sono specificati i contributi diffuso e speculare, mentre l'attenuazione è impostata per mantenere costante l'intensità e le ombre sono disabilitate per semplificare il rendering.

Infine, vengono definite le coordinate sferiche che definiscono la geolocalizzazione della simulazione utilizzando il modello terrestre WGS84 e il frame ENU. La posizione è fissata tramite latitudine e longitudine, mentre l'elevazione è posta a zero.

```

<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <world_frame_orientation>ENU</world_frame_orientation>
  <latitude_deg>47.397971057728974</latitude_deg>
  <longitude_deg>8.546163739800146</longitude_deg>
  <elevation>0</elevation>
</spherical_coordinates>

```

Oltre agli elementi statici, l'ambiente integra una componente dinamica fondamentale per la validazione degli algoritmi di percezione e inseguimento: un modello umano in movimento. Questa entità, definita come **actor_walking**, è stata inserita per fungere da target primario per il sistema di rilevamento visivo basato su YOLOv8 e per le logiche di human-following del drone.

A differenza dei modelli soggetti alla fisica standard governata da ODE, l'attore è gestito da una cinematica preimpostata e indipendente.

Si importa una mesh tridimensionale e un'animazione di camminata, regolandone il

comportamento tramite uno script in esecuzione automatica e ciclica.

```
<actor name="actor_walking">
  <skin>
    <filename>https://fuel.gazebosim.org/1.0/Mingfei/models/actor/tip/files/meshes/walk.dae</filename>
    <scale>1.0</scale>
  </skin>
  <animation name="walk">
    <filename>https://fuel.gazebosim.org/1.0/Mingfei/models/actor/tip/files/meshes/walk.dae</filename>
  </animation>
  <script>
    <loop>true</loop>
    <auto_start>true</auto_start>
    <trajectory id="0" type="walk" tension="0.6">
      <waypoint>
        <time>time</time>
        <pose>x_t y_t z_t roll_t pitch_t yaw_t</pose>
      </waypoint>
    </trajectory>
  </script>
</actor>
```

Il movimento dell'operatore è descritto da una traiettoria complessa basata su una sequenza di **waypoint spazio-temporali**, addolcita da un parametro di tensione ($tension=0.6$) che rende le rotazioni più fluide.

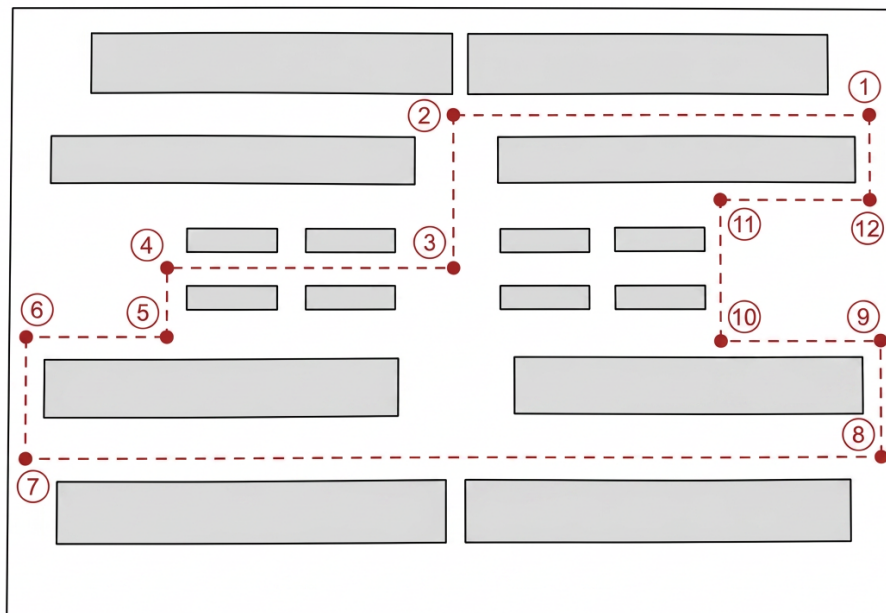


Figura 3.2: Traiettoria dell'attore dinamico attraverso gli scaffali

3.2 Modello del Quadricottero x500

Il veicolo impiegato nella simulazione è un quadricottero di tipo **Holybro x500**, descritto mediante un file URDF che definisce la struttura meccanica, le proprietà inerziali, i sensori di bordo e l'integrazione con il simulatore Gazebo.

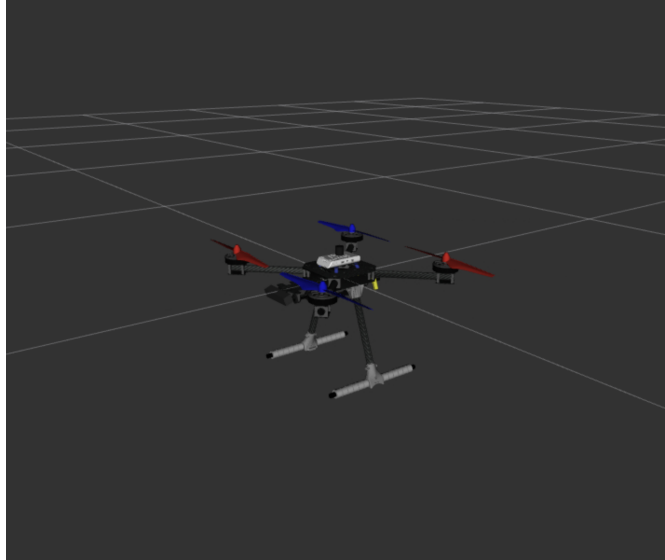


Figura 3.3: Modello del quadricottero x500 utilizzato nella simulazione.

Il modello è costituito da un corpo centrale (*base_link*) avente massa pari a 2 kg , sul quale sono montati quattro rotori, una telecamera RGB frontale e un LiDAR tridimensionale. Le proprietà dinamiche del telaio sono definite attraverso il tensore d'inerzia riportato nell'URDF, mentre ogni elica è modellata come un link indipendente collegato mediante un giunto continuo, consentendo al simulatore di calcolarne la dinamica in modo realistico.

3.2.1 Catena Cinematica

Dal punto di vista cinematica il drone costituisce un sistema ad albero con il *base_link* come riferimento principale. Tutti i sensori sono collegati tramite giunti fissi, mentre i quattro rotori sono connessi tramite giunti continui che ne permettono la rotazione attorno all'asse verticale.

La Tabella 3.1 riporta le principali trasformazioni tra i frame del robot.

Tabella 3.1: Trasformazioni principali della catena cinematica del drone.

Frame padre	Frame figlio	Tipo giunto	Trasformazione
base_link	rotor_0	continuo	$(0.174, -0.174, 0.06)$
base_link	rotor_1	continuo	$(-0.174, 0.174, 0.06)$
base_link	rotor_2	continuo	$(0.174, 0.174, 0.06)$
base_link	rotor_3	continuo	$(-0.174, -0.174, 0.06)$
base_link	camera_link	fisso	$(0.17, 0, -0.01)$
camera_link	camera_optical_link	fisso	frame ottico ROS
base_link	lidar_link	fisso	$(0, 0, 0.065)$

3.2.2 Modello dei Motori

Ogni rotore è associato al plugin **MulticopterMotorModel**, che implementa il modello aerodinamico del propulsore e converte la velocità angolare comandata nella corrispondente forza di spinta e momento torcente.

```
<plugin filename="gz-sim-multicopter-motor-model-system"
  name="gz::sim::systems::MulticopterMotorModel">
  <jointName>rotor_X_joint</jointName>
  <turningDirection>ccw/cw</turningDirection>
  <motorConstant>8.54858e-06</motorConstant>
  <momentConstant>0.016</momentConstant>
  <maxRotVelocity>1000.0</maxRotVelocity>
  <motorType>velocity</motorType>
</plugin>
```

Il medesimo plugin è istanziato per ciascuno dei quattro motori, specificando il verso di rotazione (CW o CCW), la costante di spinta, la costante di momento e la velocità massima di rotazione. Tale configurazione consente di ottenere una simulazione realistica della dinamica del quadricottero.

3.2.3 Suite Sensoriale

Il drone integra una suite sensoriale composta da una **telecamera RGB** e un **LiDAR tridimensionale**, utilizzati rispettivamente per la percezione visiva e la ricostruzione dell'ambiente circostante.

```

<sensor name="camera" type="camera">
  <update_rate>3</update_rate>
  <topic>camera</topic>
  <camera>
    <horizontal_fov>1.50098</horizontal_fov>
    <image>
      <width>640</width>
      <height>480</height>
    </image>
    <clip>
      <near>0.3</near>
      <far>15.0</far>
    </clip>
  </camera>
</sensor>

```

La telecamera acquisisce immagini RGB con risoluzione pari a 640×480 pixel, frequenza di aggiornamento di 3 Hz e campo visivo orizzontale di circa 86° . Essa costituisce il sensore principale per l'esecuzione dell'algoritmo di rilevamento persone basato su YOLOv8.

Il sensore LiDAR è installato nella parte superiore del telaio e genera una nuvola di punti tridimensionale utilizzata dagli algoritmi di navigazione.

```

<sensor name='gpu_lidar' type='gpu_lidar'>
  <gz_frame_id>lidar_link</gz_frame_id>
  <pose>0 0 0 0 0 0</pose>
  <topic>scan_cloud</topic>
  <update_rate>3</update_rate>
  <ray>
    <scan>
      <horizontal>
        <samples>360</samples>
        <resolution>1</resolution>
        <min_angle>-3.14159</min_angle>
        <max_angle>3.14159</max_angle>
      </horizontal>
      <vertical>
        <samples>16</samples>
        <resolution>1</resolution>
        <min_angle>-0.523599</min_angle>
        <max_angle>0.523599</max_angle>
      </vertical>
    </scan>
  </ray>
</sensor>

```

```

    <range>
      <min>0.4</min>
      <max>15.0</max>
      <resolution>0.01</resolution>
    </range>
  </ray>
  <always_on>true</always_on>
  <visualize>true</visualize>
</sensor>

```

Il LiDAR ha un campo visivo di 360° sul piano orizzontale e $\pm 30^\circ$ su quello verticale. Il sensore acquisisce nuvole di punti fino a 15 m di distanza con una frequenza di 3 Hz, permettendo il rilevamento degli ostacoli e la ricostruzione tridimensionale dell'ambiente.

3.2.4 Plugin di simulazione

Oltre al modello fisico del drone, l'URDF integra ulteriori plugin dedicati alla pubblicazione dello stato del robot.

```

<plugin filename="gz-sim-odometry-publisher-system"
  name="gz::sim::systems::OdometryPublisher">
  <odom_publish_frequency>50</odom_publish_frequency>
  <publish_tf>true</publish_tf>
</plugin>

<plugin filename="gz-sim-joint-state-publisher-system"
  name="gz::sim::systems::JointStatePublisher">
  <joint_name>rotor_0_joint</joint_name>
  <joint_name>rotor_1_joint</joint_name>
  <joint_name>rotor_2_joint</joint_name>
  <joint_name>rotor_3_joint</joint_name>
</plugin>

```

L'**OdometryPublisher** pubblica la posa e l'odometria del drone alla frequenza di 50 Hz, generando inoltre le trasformazioni TF necessarie all'integrazione con ROS 2. Il plugin **JointStatePublisher** rende invece disponibili gli stati dei quattro giunti dei rotori, permettendo la sincronizzazione con gli altri nodi del sistema.

Capitolo 4

Progettazione e Implementazione del Sistema di Controllo del Volo

La stabilità e la precisione di manovra del velivolo sono garantite dall'implementazione di un nodo di controllo custom denominato `FullDroneController`. Quest'ultimo adotta un'architettura ad anelli in cascata (cascaded control), integrando azioni proporzionali-integrali-derivative (PID) accoppiate a termini predittivi di feedforward cinematico per l'annullamento del ritardo di fase durante l'inseguimento di traiettorie dinamiche.

4.1 Architettura di Controllo

La dinamica di un quadricottero è intrinsecamente non lineare, instabile e a forte accoppiamento multivariabile. Per semplificare il problema di regolazione, il sistema software è stato strutturato scindendo le componenti traslazionali (lente) da quelle rotazionali (veloci) attraverso una topologia di controllo in cascata organizzata su due livelli macroscopici:

1. **Anello Esterno (Outer Loop - Controllo di Posizione):** Opera a frequenza cinematica basandosi sui riferimenti spaziali. Riceve gli errori di posizionamento cartesiano sul piano orizzontale e calcola l'assetto angolare (rollio e beccheggio) necessario affinché la spinta totale del drone si inclini nella direzione dello spostamento desiderato.
2. **Anello Interno (Inner Loop - Controllo di Assetto e Altitudine):** Lavora ad alta frequenza ed è agganciato direttamente alla ricezione dei dati odometrici del topic `/odom`. Questo anello stabilizza l'orientamento del velivolo inseguendo i setpoint di rollio e beccheggio generati dall'anello esterno, oltre a regolare in modo indipendente l'altitudine (Z) e l'imbardata (Yaw).

Per assicurare la massima accuratezza temporale ed evitare il rigetto di messaggi o sfasamenti nei buffer di trasformazione (TF) all'interno di RViz, l'intero sistema è configurato per operare sfruttando rigorosamente il tempo della simulazione (`use_sim_time = True`), sincronizzando il clock dei nodi ROS 2 direttamente con il risolutore fisico di Gazebo.

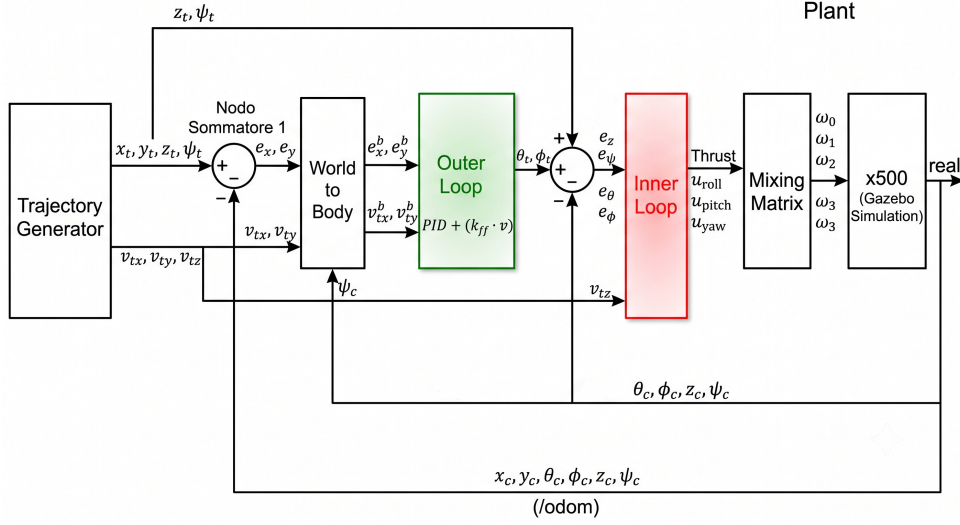


Figura 4.1: Diagramma a Blocchi dell'architettura di controllo

4.1.1 Formalizzazione Matematica

Per comprendere appieno la logica operativa del nodo `FullDroneController`, occorre partire dalla formulazione ideale di un regolatore Proporzionale-Integrale-Derivativo (PID) in tempo continuo, la cui azione di controllo $u(t)$ è definita dalla combinazione lineare di tre contributi distinti applicati all'errore istantaneo $e(t)$:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

All'interno della classe `PIDController`, questa formulazione continua viene discretizzata per l'esecuzione a tempo discreto in corrispondenza di ogni ciclo di callback dell'odometria, dove l'intervallo di campionamento non è necessariamente costante ma varia in base al passo temporale reale Δt calcolato istante per istante.

Come primo controllo, se un pacchetto dati potrebbe arrivare in ritardo o presenta un timestamp duplicato, l'intervallo temporale risulta nullo o negativo; quindi l'algoritmo forza l'uscita a zero:

$$\text{Se } \Delta t \leq 0.0 \implies u_k = 0.0$$

Tale verifica previene l'insorgere di divisioni per zero durante il calcolo della componente derivativa.

Successivamente, il termine integrale viene calcolato mediante approssimazione rettangolare in avanti:

$$I_{raw,k} = I_{k-1} + (e_k \cdot \Delta t)$$

Per neutralizzare il noto e distruttivo fenomeno dell'integrazione windup la variabile integrale viene sottoposta a una limitazione geometrica rigida (clamping).

$$I_k = \max(-50.0, \min(50.0, I_{raw,k}))$$

Parallelamente, l'azione derivativa è ricavata applicando il metodo delle differenze finite all'indietro:

$$D_k = \frac{e_k - e_{k-1}}{\Delta t}$$

Al termine del calcolo, si avrà:

$$u_{raw,k} = (K_p \cdot e_k) + (K_i \cdot I_k) + (K_d \cdot D_k)$$

Infine il comando grezzo globale viene a sua volta limitato entro i confini operativi dell'attuatore virtuale, definiti dai parametri di inizializzazione `min_out` e `max_out`:

$$u_k = \max(\min_out, \min(\max_out, u_{raw,k}))$$

4.2 Anello Esterno

L'anello esterno si occupa di tradurre l'errore di posizione globale $(\Delta x, \Delta y)$ nei riferimenti di inclinazione necessari a spostare il baricentro del drone nella direzione desiderata.

Poiché i comandi di rollio e beccheggio operano localmente rispetto al *Body Frame*, le componenti dell'errore calcolate nel *World Frame* devono essere proiettate vettorialmente sfruttando l'angolo di imbardata corrente del velivolo ψ :

$$\begin{bmatrix} e_{x,body} \\ e_{y,body} \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} x_{target} - x_{curr} \\ y_{target} - y_{curr} \end{bmatrix}$$

Allo stesso modo, le velocità ideali subiscono la medesima trasformazione geometrica:

$$\begin{bmatrix} v_{tx,body} \\ v_{ty,body} \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} v_{x,target} \\ v_{y,target} \end{bmatrix}$$

A questo punto:

$$u_x = \text{PID}_x(e_{x,body}, \Delta t) + (k_{ff,xy} \cdot v_{tx,body})$$

$$u_y = \text{PID}_y(e_{y,body}, \Delta t) + (k_{ff,xy} \cdot v_{ty,body})$$

dove l'azione predittiva di *feedforward* con il termine $k_{ff,xy}$ nieta immediatamente nei comandi la pendenza della traiettoria futura, anticipando dinamicamente il movimento ed eliminando di fatto il ritardo di inseguimento transitorio.

Queste uscite virtuali, limitate per sicurezza entro ± 0.25 rad per evitare repentine cadute di portanza causate da eccessive inclinazioni, diventano i setpoint di assetto del velivolo. Applicando le regole della meccanica del volo per un sistema di riferimento destrorso, un'accelerazione longitudinale richiede un beccheggio concorde, mentre una spinta laterale destra necessita di un rollio di segno discorde:

$$\theta_{\text{target}} = u_x \quad \text{e} \quad \phi_{\text{target}} = -u_y$$

4.3 Anello Interno

L'anello di controllo interno chiude il ciclo elaborando i setpoint di assetto e garantendo al contempo il corretto mantenimento dell'altitudine e della prua direzionale.

Nel trattare l'asse di imbardata, sorge un'anomalia geometrica legata alla rappresentazione angolare: un errore calcolato per differenza algebrica potrebbe suggerire una rotazione inefficiente se il target attraversa la discontinuità di $\pm\pi$ rad. Per ovviare a ciò e instradare sempre il drone sul percorso angolare più breve, l'errore di imbardata viene normalizzato circolarmente:

$$e_\psi = \text{atan2}(\sin(\psi_{\text{target}} - \psi_{\text{curr}}), \cos(\psi_{\text{target}} - \psi_{\text{curr}}))$$

Parallelamente, il controllo dell'altitudine (Z) viene gestito da un regolatore PID dedicato che compensa gli scostamenti verticali.

A completamento dell'architettura predittiva introdotta per i moti planari, anche sull'asse verticale il sistema incorpora un guadagno di *feedforward* ($k_{ff,z}$) proporzionale alla velocità ascensionale target. In questo modo, l'azione di controllo si compone di una componente reattiva e di una anticipatoria, abbattendo il ritardo di inseguimento durante le variazioni di quota (salite o discese):

$$u_{\text{alt}} = \text{PID}_{\text{alt}}(z_{\text{target}} - z_{\text{curr}}, \Delta t) + (k_{ff,z} \cdot v_{z,\text{target}})$$

L'uscita di questo regolatore viene infine sommata a una forza propulsiva statica di bilanciamento (`base_thrust`), indispensabile per vincere la forza peso del velivolo a riposo (hovering). Il calcolo teorico di questo parametro deriva dall'equilibrio delle forze lungo l'asse verticale.

Analizzando il modello fisico, la massa totale del drone m è data dalla somma del corpo centrale e dei quattro rotori:

$$m = m_{base} + 4 \cdot m_{rotore} = 2.0 + 4 \cdot 0.01607 \approx 2.064 \text{ kg}$$

Sottoposto all'accelerazione di gravità simulata nell'ambiente ($g = 9.81 \text{ m/s}^2$), il velivolo genera una forza peso W :

$$W = m \cdot g \approx 2.064 \cdot 9.81 \approx 20.23 \text{ N}$$

Dal modello aerodinamico del plugin `MulticopterMotorModel`, la spinta generata da un singolo rotore è proporzionale al quadrato della sua velocità angolare tramite la costante di spinta del motore C_T :

$$T_i = C_T \cdot \omega_i^2$$

dove $C_T = 8.54858 \times 10^{-6}$. Imponendo l'equilibrio traslazionale per il quale la spinta combinata dei quattro rotori deve bilanciare esattamente il peso ($4 \cdot T_i = W$), si ottiene la velocità angolare nominale di hovering ω_{hover} :

$$\omega_{hover} = \sqrt{\frac{m \cdot g}{4 \cdot C_T}} = \sqrt{\frac{20.23}{4 \cdot 8.54858 \cdot 10^{-6}}} = 769.16 \text{ rad/s}$$

Il valore teorico ottenuto viene quindi impiegato come termine di polarizzazione costante all'interno dell'algoritmo, sostituendo approcci empirici e garantendo l'equilibrio a riposo:

$$Thrust = \omega_{hover} + u_{alt}$$

Infine, i PID di stabilizzazione del Rollio (ϕ) e del Beccheggio (θ) inseguono gli angoli generati dall'anello esterno e producono a loro volta sollecitazioni virtuali in termini di momenti torcenti u_{roll} e u_{pitch} .

Un ulteriore PID stabilizza l'imbardata erogando il comando u_{yaw} .

4.4 Mixing Matrix e Attuazione

L'ultima e cruciale operazione matematica consiste nella trasduzione delle quattro forze e momenti virtuali ($Thrust, u_{roll}, u_{pitch}, u_{yaw}$) in comandi da destinare al plant fisico, ovvero alla dinamica dei rotori. Il drone x500 adotta una configurazione geometrica ad "X", in cui i rotori 0 e 1 ruotano in senso antiorario (CCW), mentre i rotori 2 e 3 ruotano in senso orario (CW).

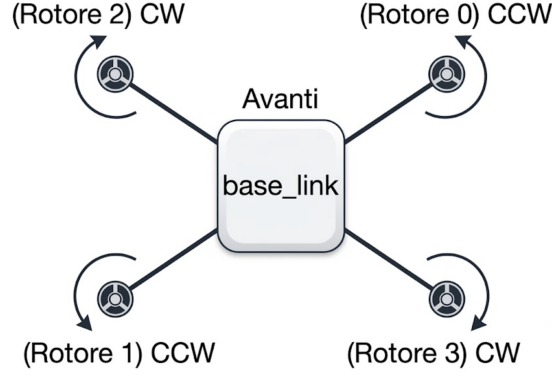


Figura 4.2: Schema geometrico di rotazione dei rotori per la configurazione ad 'X'

La ripartizione del carico di lavoro sulle quattro eliche viene formalizzata attraverso la matrice di allocazione del controllo (o matrice di mixing). Il vettore delle velocità angolari target dei motori Ω si ottiene moltiplicando la matrice di configurazione M per il vettore dei comandi virtuali U :

$$\Omega = M \cdot U$$

Espandendo l'equazione con i parametri specifici per la configurazione del quadricottero x500 implementata nel nodo di controllo, si ottiene:

$$\begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \begin{bmatrix} 1 & -1 & -1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \text{Thrust} \\ u_{\text{roll}} \\ u_{\text{pitch}} \\ u_{\text{yaw}} \end{bmatrix}$$

Come atto di sicurezza finale per preservare la stabilità termo-meccanica dei modelli motorizzati nella simulazione, ciascuna componente del vettore Ω viene forzata all'interno dei limiti fisicamente supportati dai rotori. La funzione di saturazione tronca le velocità angolari nel range ammissibile compreso tra 0.0 e 1000.0 rad/s:

$$\omega_i = \max(0.0, \min(1000.0, \omega_i)) \quad \forall i \in \{0, 1, 2, 3\}$$

La Tabella 4.1 guadagna dei PID utilizzati nella progettazione dell'architettura di controllo.

Tabella 4.1: Parametri di configurazione dei regolatori PID e Feedforward

Variabile Controllata	K_p	K_i	K_d	K_{ff}	Min Out	Max Out
Anello Esterno (Outer Loop - Posizione)						
Posizione X	0.15	0.0	0.1	0.1	-0.25	0.25
Posizione Y	0.15	0.0	0.1	0.1	-0.25	0.25
Anello Interno (Inner Loop - Assetto e Altitudine)						
Altitudine (Z)	15.0	0.05	45.0	0.5	-150.0	150.0
Rollio (ϕ)	35.0	0.01	8.0	-	-80.0	80.0
Beccheggio (θ)	35.0	0.01	8.0	-	-80.0	80.0
Imbardata (ψ)	8.0	0.01	13.0	-	-50.0	50.0

Capitolo 5

Costruzione della Mappa (RTAB-Map)

All'interno dell'architettura di navigazione autonoma sviluppata, il modulo di mappatura riveste un ruolo nevralgico per l'esplorazione dell'ambiente logistico e la consapevolezza spaziale del velivolo. Il compito di costruire una rappresentazione spaziale coerente, densa e persistente del magazzino è stato affidato al framework ROS 2 RTAB-Map (Real-Time Appearance-Based Mapping). Sebbene originariamente concepito per l'elaborazione di flussi visivi RGB-D, in questo progetto il core algoritmico è stato accuratamente configurato per operare come un sistema puro di Graph-SLAM basato su nuvole di punti tridimensionali.

5.1 Architettura e Sincronizzazione Sensoriale

L'implementazione algoritmica scelta disabilita esplicitamente le pipeline di elaborazione video, ponendo a **False** i parametri `subscribe_rgb` e `subscribe_depth`.

Il modulo di SLAM si affida in via esclusiva al flusso di dati proveniente dal sensore LiDAR 3D di bordo, iscrivendosi al topic `/scan_cloud` alla frequenza di aggiornamento di 3 Hz.

Affinché la proiezione dei punti nello spazio globale avvenga senza distorsioni, il sistema deve gestire con estremo rigore la catena delle trasformazioni cinematiche (TF Tree). Il file di lancio configura RTAB-Map per coordinare la gerarchia dei tre frame principali del sistema:

- `base_link`: Il riferimento locale solidale al baricentro del quadricottero.

- **odom**: Il sistema di riferimento inerziale locale generato dall'odometria del simulatore fisico. Essendo soggetto a inevitabili derive numeriche nel corso del tempo, questo frame garantisce fluidità a breve termine ma perde precisione globale.
- **map**: Il sistema di riferimento globale e assoluto dell'ambiente logistico

Il nodo RTAB-Map elabora i dati sensoriali e calcola dinamicamente la trasformazione spaziale tra **map** e **odom**, compensando in tempo reale l'errore di deriva odometrica accumulato dal drone. Per garantire l'assoluto allineamento spazio-temporale tra la posa del drone e la scansione laser, il parametro `odom_sensor_sync` è impostato a **True**. Questa policy di sincronizzazione forza i filtri dei messaggi ROS 2 ad attendere e accoppiare esclusivamente i pacchetti `/odom` e `/scan_cloud` dotati del medesimo timestamp, evitando il fenomeno dello "skewing" (distorsione geometrica della nuvola) causato dal moto traslazionale del velivolo durante il tempo di campionamento del sensore.

5.2 Pre-Processing e Generazione Voxel Grid

Prima di essere fusa all'interno del grafo dello SLAM, la nuvola di punti grezza subisce una **pipeline di pre-elaborazione** per ottimizzare l'occupazione in memoria RAM e abbattere il carico computazionale sulla CPU. Il sistema attiva il filtro `Cloud/FilterNaN: true` per scartare misurazioni non valide, quali i raggi riflessi all'infinito o completamente assorbiti dall'ambiente, e applica una decimazione spaziale (`Cloud/Decimation: 2`). Quest'ultimo parametro sottocampiona la matrice dei punti dimezzandone la densità originaria, preservando intatta la topologia macroscopica degli ostacoli ma riducendo drasticamente il costo dei calcoli successivi.

Per consentire la discretizzazione spaziale richiesta dall'algoritmo di pianificazione A^* , RTAB-Map è stato configurato per allocare dinamicamente una Occupancy Grid 3D basata su struttura dati OctoMap. Attraverso i parametri `RGBD/CreateOccupancyGrid: true` e `Grid/3D: true`, il nodo core costruisce una struttura volumetrica a voxel complessa ed efficiente, eliminando la necessità di eseguire nodi ROS 2 aggiuntivi (come `octomap_server`).

I parametri operativi dell'algoritmo di proiezione volumetrica sono stati calibrati sulle specifiche costruttive del LiDAR:

- **Gestione Superfici**: La griglia viene estratta unicamente dalla geometria della scansione laser (`Grid/Sensor: 0` e `Grid/FromDepth: false`). Essendo il da-

to di partenza una nuvola geometricamente sparsa e priva di texture, l'algoritmo di segmentazione delle superfici basato sul calcolo delle normali viene disattivato (`Grid/NormalsSegmentation: false`), risparmiando gravosi cicli di calcolo solitamente riservati alle telecamere.

- **Risoluzione e Portata:** La dimensione laterale del singolo voxel cubico è imposta a 0.25 m (`Grid/CellSize: 0.25`), un compromesso ideale per garantire il passaggio sicuro nei corridoi stretti del magazzino senza causare l'esplosione esponenziale della memoria necessaria alla memorizzazione dell'OctoMap. I punti vengono integrati fino a una distanza radiale massima di 15.0 m, che definisce l'orizzonte sensoriale del velivolo.
- **Ray Tracing Dinamico:** L'approccio volumetrico non si limita all'accumulo additivo dei punti di impatto, ma gestisce la probabilità di occupazione dello spazio libero tramite il parametro `Grid/RayTracing: true`. RTAB-Map applica un algoritmo di ispezione spaziale che traccia vettorialmente l'intero percorso del raggio laser, riducendo la probabilità di occupazione (e quindi marcando come "liberi") tutti i voxel attraversati prima dell'ostacolo fisico. Questa operazione è fondamentale per l'inseguimento del target umano: man mano che la persona cammina tra gli scaffali, il Ray Tracing si occupa di "cancellare" i voxel precedentemente occupati dal suo corpo, prevenendo la formazione di barriere fantasma che ostacolerebbero irrimediabilmente il calcolo delle traiettorie.

5.3 Gestione del Grafo SLAM e Policy di Aggiornamento

Il framework opera secondo il paradigma del Graph-Based SLAM. La persistenza a lungo termine della topologia ambientale è governata dall'attivazione della memoria incrementale (`Mem/IncrementalMemory: true`), che ordina al sistema di accumulare coerentemente i dati sotto forma di Nodi (posizioni stimate) e Link (vincoli spaziali derivanti dall'odometria e dalle misurazioni LiDAR) durante l'intera missione esplorativa.

La frequenza di elaborazione globale, comprensiva della verifica dei "Loop Closure" (riconoscimento di luoghi già visitati per annullare la deriva accumulata), è fissata a 1 Hz. La strategia di registrazione utilizzata a livello core per allineare le nuvole di punti è quella standard spaziale definita dal parametro `Reg/Strategy: 0`.

La crescita computazionale incontrollata del grafo SLAM viene mitigata da soglie di aggiornamento (Keyframe Extraction) altamente restrittive. Un nuovo nodo viene registrato in memoria solo se il drone ha compiuto uno scostamento cinematico sufficientemente rilevante rispetto all'ultimo fotogramma salvato:

- Variazione traslazionale lineare di almeno 0.05 m (RGBD/LinearUpdate: 0.05).
- Variazione angolare dell'assetto di almeno 0.01 rad (RGBD/AngularUpdate: 0.01).

Questi vincoli, particolarmente stringenti, assicurano che le frequenti ma microscopiche manovre di correzione del beccheggio e rollio per mantenere l'hovering tra gli scaffali vengano convertite in informazioni utili alla mappatura, mantenendo altissima la reattività dell'aggiornamento spaziale.

5.4 Topic ROS 2 e Visualizzazione

L'elaborazione asincrona del nodo produce un'estesa gerarchia di strutture dati, che vengono instradate sulla topologia ROS 2 per alimentare il controllo di volo e le interfacce diagnostiche. Tra i numerosi topic esposti dall'istanza, i principali si dividono per caso d'uso:

1. **Interfacce Operative** (Pianificazione): Il sistema espone il topic `/octomap_occupied_space`, contenente esclusivamente il *PointCloud* relativo ai voxel riconosciuti come ostacoli statici. Questo flusso dati viene sottoscritto direttamente dal nodo `planner.py` per popolare in tempo reale la griglia dei costi tridimensionale ed eseguire la ricerca euristica A^* .
2. **Topologia del Grafo SLAM**: I topic `/mapGraph` e `/mapData` trasportano le informazioni matematiche della rete spaziale, inclusi i vincoli odometrici ottimizzati dal risolutore interno di RTAB-Map.
3. **Flussi Diagnostici per la Visualizzazione**: Per consentire l'ispezione visiva, il modulo espone `/rtabmap/octomap_full` (la struttura completa della memoria ad albero con voxel liberi e occupati), `/rtabmap/octomap_binary` (la mappa binaria dei soli ostacoli) e il cloud aggregato su `/cloud_map`.

Per l'ispezione della mappatura e del grafo durante la simulazione fisica, il sistema istanzia direttamente il nodo nativo `rtabmap_viz`, configurato nel file di lancio per ricevere i medesimi remapping e parametri del nodo core.

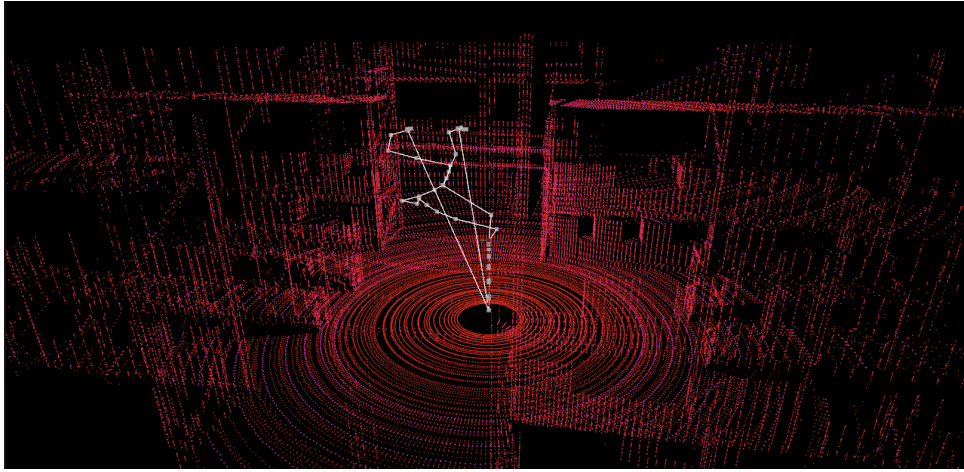


Figura 5.1: Interfaccia diagnostica di RTAB-Map

L'interfaccia offre una rappresentazione visiva diretta della memoria interna dell'algoritmo. In rosso è visibile l'aggregazione delle nuvole di punti 3D (Point Cloud) acquisite dal LiDAR, che delineano chiaramente le scaffalature del magazzino e il pattern di scansione concentrico sul pavimento.

Capitolo 6

Pianificazione della Traiettoria e Navigazione Autonoma 3D

Il sottosistema di navigazione rappresenta il nucleo decisionale del velivolo autonomo. L'architettura sviluppata si propone di superare i limiti della classica navigazione punto-punto (stop-and-go), introducendo una scomposizione a due stadi: un pianificatore spaziale globale operante su rappresentazione discreta e un generatore di traiettoria continuo basato su curve polinomiali parametriche e interpolazione sferica.

6.1 Architettura Generale

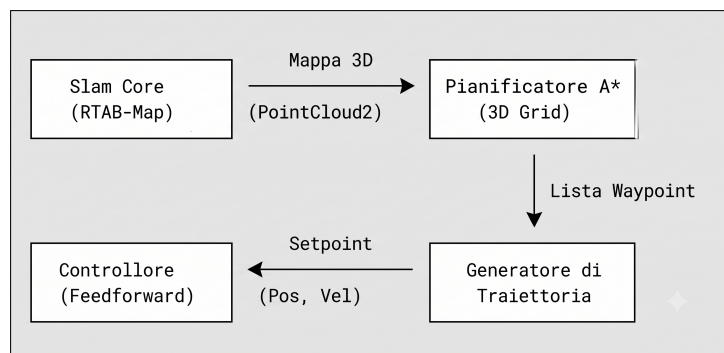


Figura 6.1: Schema dei flussi operativi tra i moduli

Il flusso di controllo della navigazione si articola secondo uno schema gerarchico a cascata. Il modulo di mappatura tridimensionale fornisce la geometria dell'ambiente sotto forma di nuvola di punti strutturata. Su questa base, il nodo `planner.py` calcola un cammino privo di collisioni composto da una sequenza discreta di nodi spaziali e orientamenti ideali. Tale cammino (pubblicato come lista di pose) non viene inviato direttamente all'unità di attuazione, ma viene preso in carico dal nodo `trajectory_generator.py`, che ne esegue

l'interpolazione e la parametrizzazione temporale, calcolando istante per istante le posizioni, le velocità e gli assetti ideali necessari ad alimentare l'azione di feedforward del controllore di volo.

6.2 Discretizzazione Spaziale e Gestione della Voxel Grid

Per consentire l'applicazione di algoritmi di ricerca grafi in uno spazio tridimensionale continuo, l'ambiente circostante viene discretizzato all'interno di una **Voxel Grid**.

Il nodo riceve dal modulo Rtabmap la nuvola di punti corrispondente allo spazio occupato.

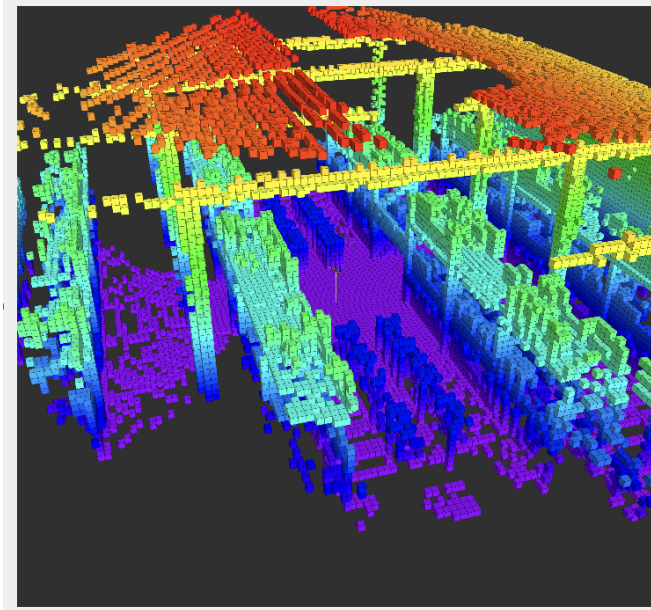


Figura 6.2: Rappresentazione dello spazio occupato

La discretizzazione e la mappatura bidirezionale tra il World Frame continuo e il Grid Frame discreto sono regolate dalle seguenti relazioni matematiche:

$$i = \lfloor \frac{x}{L_{\text{cell}}} \rfloor, \quad j = \lfloor \frac{y}{L_{\text{cell}}} \rfloor, \quad k = \lfloor \frac{z}{L_{\text{cell}}} \rfloor$$

$$x = (i + 0.5) \cdot L_{\text{cell}}, \quad y = (j + 0.5) \cdot L_{\text{cell}}, \quad z = (k + 0.5) \cdot L_{\text{cell}}$$

dove L_{cell} rappresenta la dimensione lineare del singolo voxel equatoriale.

Al fine di garantire l'incolumità strutturale del quadricottero durante le manovre di volo in prossimità di ostacoli o pareti del magazzino, viene implementato un algoritmo di rigonfiamento geometrico (**Voxel Inflation**). Ciascun punto rilevato come ostacolo non occupa una singola cella, ma propaga la propria condizione di occupazione all'interno di un intorno sferico discreto di raggio r_{inflate} (`inflate_cells`).

L'insieme delle celle occupate O_{total} è definito formalmente da:

$$O_{\text{total}} = \{(i + di, j + dj, k + dk) \mid (i, j, k) \in O_{\text{raw}} \wedge \max(|di|, |dj|, |dk|) \leq r_{\text{inflate}}\}$$

6.3 Weighted A^* 3D

Calcolata la configurazione della griglia, la determinazione del percorso ottimale privo di ostacoli viene affidata a un algoritmo A^* tridimensionale.

La ricerca si sviluppa esplorando uno spazio a 26 gradi di connettività vicinale, considerando per ciascun nodo sia i movimenti ortogonali che quelli diagonali nello spazio cubico circostante:

$$\text{Neighbours}(i, j, k) = \{(i + di, j + dj, k + dk) \mid di, dj, dk \in \{-1, 0, 1\} \setminus (0, 0, 0)\}$$

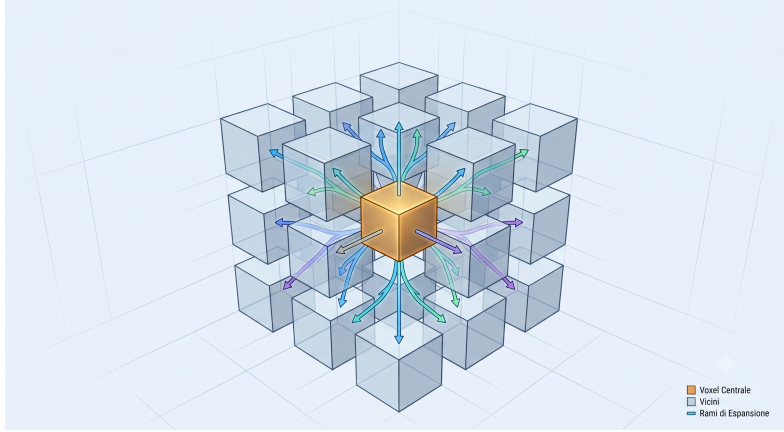


Figura 6.3: Rappresentazione geometrica dello spazio di ricerca

Il costo di transizione $c(n, n')$ tra un nodo corrente e un suo vicino viene calcolato come la distanza euclidea spaziale inter-cella:

$$c(n, n') = \sqrt{di^2 + dj^2 + dk^2}$$

La valutazione complessiva dei nodi inseriti nella coda con priorità è regolata dalla funzione di costo totale pesata:

$$f(n) = g(n) + W \cdot h(n) \tag{6.1}$$

dove $g(n)$ rappresenta il costo esatto accumulato dal nodo di partenza fino al nodo corrente n , mentre $h(n)$ è la funzione euristica definita come la distanza euclidea residua rispetto al target finale:

$$h(n) = \sqrt{(i_{\text{goal}} - i_n)^2 + (j_{\text{goal}} - j_n)^2 + (k_{\text{goal}} - k_n)^2}$$

Il fattore moltiplicativo W viene impostato a 1.5, configurando l'algoritmo come un Weighted A^* . Questa estensione introduce una sub-ottimalità controllata sul cammino geometrico finale in cambio di una drastica riduzione dello spazio di stato esplorato e del numero di espansioni totali nella coda. Ciò permette al nodo di convergere rapidamente alla soluzione, abbattendo il carico computazionale sulla CPU.

La procedura completa di pianificazione viene riportata nell'Algoritmo 1.

La prima fase consiste nella trasformazione delle coordinate metriche del punto iniziale e del punto obiettivo nella corrispondente rappresentazione discreta della griglia.

Successivamente viene eseguita l'esplorazione dello spazio di configurazione attraverso una coda di priorità, nella quale ogni nodo viene valutato mediante la funzione di costo pesata (6.1).

Durante la ricerca viene mantenuta una struttura dati dei predecessori (`cameFrom`), necessaria per ricostruire il percorso una volta raggiunto il nodo obiettivo. Terminata la fase di esplorazione, la sequenza discreta di celle attraversate viene riconvertita nello spazio metrico calcolando il centro geometrico di ogni voxel. Tuttavia, prima di essere utilizzata dal modulo di generazione della traiettoria continua, questa lista grezza di waypoint subisce una rigorosa pipeline di pulizia geometrica al fine di garantirne la stabilità cinematica.

Algorithm 1 Weighted A^* 3D

Require: Posizione iniziale P_s , posizione finale P_g , voxel grid

Ensure: Sequenza ordinata di waypoint metrici

```
1:  $start \leftarrow WorldToGrid(P_s)$ 
2:  $goal \leftarrow WorldToGrid(P_g)$ 
3:  $Open \leftarrow PriorityQueue()$ 
4: Inserisci  $start$  in  $Open$  con priorità 0
5:  $cameFrom[start] \leftarrow \emptyset$ 
6:  $gCost[start] \leftarrow 0$ 
7: while  $Open$  non è vuota do
8:    $current \leftarrow PopMin(Open)$ 
9:   if  $current = goal$  then
10:    break
11:   end if
12:   for all  $neighbor \in Neighbors_{26}(current)$  do
13:     if  $neighbor$  è occupato then
14:       continue
15:     end if
16:      $stepCost \leftarrow \sqrt{di^2 + dj^2 + dk^2}$ 
17:      $newCost \leftarrow gCost[current] + stepCost$ 
18:     if  $neighbor \notin gCost$  or  $newCost < gCost[neighbor]$  then
19:        $gCost[neighbor] \leftarrow newCost$ 
20:        $cameFrom[neighbor] \leftarrow current$ 
21:        $h \leftarrow EuclideanDistance(neighbor, goal)$ 
22:        $f \leftarrow newCost + 1.5 \cdot h$ 
23:       Inserisci  $(neighbor, f)$  in  $Open$ 
24:     end if
25:   end for
26: end while
27: if  $goal \notin cameFrom$  then
28:   return Nessun percorso disponibile
29: end if
30:  $path \leftarrow []$ 
31:  $node \leftarrow goal$ 
32: while  $node \neq \emptyset$  do
33:   Inserisci  $node$  all'inizio di  $path$ 
34:    $node \leftarrow cameFrom[node]$ 
35: end while
36:  $trajectory_{raw} \leftarrow []$ 
37: for all  $(i, j, k) \in path$  do
38:    $x \leftarrow (i + 0.5)L_{cell}$ 
39:    $y \leftarrow (j + 0.5)L_{cell}$ 
40:    $z \leftarrow (k + 0.5)L_{cell}$ 
41:   Aggiungi  $(x, y, z)$  a  $trajectory_{raw}$ 
42: end for
43:  $trajectory_{simplify} \leftarrow SimplifyPath(trajectory_{raw})$ 
44:  $trajectory_{min\_space} \leftarrow EnforceMinSpacing(trajectory_{simplify})$ 
45:  $trajectory \leftarrow MergeCollinear(trajectory_{min\_space})$ 
46: return  $trajectory$ 
```

Un aspetto fondamentale è il **+0.5** nella conversione finale; moltiplicare semplicemente l'indice della griglia per la dimensione della cella porterebbe esattamente sullo spigolo

inferiore sinistro del voxel. Aggiungere 0.5 assicura che il waypoint spaziale venga calcolato esattamente nel centro geometrico del cubo libero, tenendo il drone il più lontano possibile dalle pareti della cella virtuale.

6.3.1 Semplificazione Greedy

La sottosezione `simplify_path` implementa una strategia di semplificazione greedy atta a ridurre la complessità computazionale del percorso, eliminando i waypoint che giacciono su segmenti privi di ostacoli.

Il modulo analizza la lista dei nodi mantenendo un **anchor point** P_{anchor} e tentando di congiungerlo direttamente con i punti successivi P_{i+1} . La validità geometrica di tale collegamento è delegata a una variante 3D dell'**algoritmo di Bresenham**, che verifica l'assenza di collisioni lungo il segmento.

Per orientare il tracciamento nello spazio, il si definiscono gli incrementi direzionali unitari, necessari per gestire il movimento in qualsiasi direzione della griglia:

$$s_x = \text{sign}(x_2 - x_1), \quad s_y = \text{sign}(y_2 - y_1), \quad s_z = \text{sign}(z_2 - z_1)$$

L'algoritmo avanza lungo l'asse predominante (quello con lo spostamento maggiore) e utilizza due variabili di errore, p_1 e p_2 , per decidere quando incrementare le coordinate secondarie e mantenere la retta ideale. Queste variabili, che misurano lo scostamento cumulativo dalla traiettoria teorica, sono inizializzate come:

$$p_1 = 2\Delta y - \Delta x, \quad p_2 = 2\Delta z - \Delta x$$

Ad ogni passo dell'avanzamento, le variabili di errore vengono aggiornate. Quando uno degli errori diventa positivo, il sistema incrementa la coordinata corrispondente e corregge l'errore sottraendo $2\Delta x$.

Contemporaneamente, a ogni iterazione, viene eseguito il test `is_occupied(cx, cy, cz)` per interrogare lo stato di occupazione dei voxel attraversati.

Se la funzione rileva una collisione, il segmento teorico è ostruito; quindi, l'algoritmo interrompe la verifica, conserva il punto intermedio P_i e lo elegge a nuovo P_{anchor} .

Al contrario, se il test `is_occupied` risulta negativo per l'intero segmento, la rotta è garantita libera da ostacoli. In questo caso, il punto intermedio P_i viene rimosso definitivamente dal percorso, ottimizzando la sequenza dei nodi per il successivo modulo di interpolazione spline.

6.3.2 Filtro di Spaziatura Minima

Il secondo stadio della pipeline funge da vero e proprio filtro passa-basso spaziale, necessario per prevenire la degenerazione numerica delle spline cubiche `enforce_min_spacing`.

Il filtro agisce preservando il punto di partenza e analizzando sequenzialmente i nodi successivi: un punto P_i viene mantenuto nel percorso solo se rispetta una distanza euclidea minima Δ_{min} rispetto all'ultimo punto validato P_{last} :

$$\sqrt{(x_i - x_{last})^2 + (y_i - y_{last})^2 + (z_i - z_{last})^2} \geq \Delta_{min}$$

6.3.3 Fusione Collineare

L'ultimo stadio della pipeline ottimizza il numero di gradi di libertà del percorso, eliminando i waypoint posizionati su traiettorie rettilinei o con variazioni angolari trascurabili `merge_collinear`.

L'algoritmo esamina le triplette di nodi $(P_{prev}, P_{curr}, P_{next})$ per valutare se la deviazione introdotta da P_{curr} sia geometricamente significativa.

L'analisi si basa sulla scomposizione vettoriale del percorso; si definiscono i vettori di direzione dei due segmenti adiacenti come $\vec{v}_1 = P_{curr} - P_{prev}$ e $\vec{v}_2 = P_{next} - P_{curr}$. La curvatura del tracciato viene quindi determinata calcolando il coseno dell'angolo compreso θ tramite il prodotto scalare normalizzato:

$$\cos(\theta) = \frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \|\vec{v}_2\|}$$

Il valore ottenuto viene confrontato con la soglia $\cos(\theta_s)$. Se $\cos(\theta) > \cos(\theta_s)$ la variazione di direzione è considerata trascurabile, P_{curr} è etichettato come collineare e rimosso dalla sequenza.

Se, al contrario, il punto viene riconosciuto come un vertice significativo e preservato per definire correttamente la geometria della curva.

6.4 Generazione di Traiettorie e Parametrizzazione Temporale

L'invio diretto dei waypoint discreti al controllore di volo comporterebbe un moto discontinuo e inefficiente, caratterizzato da repentine decelerazioni e arresti in corrispondenza di ciascun vertice. Per ovviare a questo problema, il nodo intercetta l'intera sequenza generata dal pianificatore A^* e la converte in una traiettoria temporale continua di classe C^2 .

6.4.1 Pre-filtraggio Spaziale e Assegnazione Temporale

Sia $P_{raw} = \{p_0, p_1, \dots, p_N\}$ l'insieme iniziale dei waypoint tridimensionali, dove p_0 è sovrascritto dall'esatta posizione odometrica istantanea del quadricottero per azzerare l'errore di inseguimento iniziale.

Come fase preliminare, l'algoritmo effettua un filtraggio spaziale rimuovendo i nodi successivi p_k la cui distanza euclidea dal nodo validato precedente p_{k-1} risulti inferiore a una soglia di tolleranza $\epsilon = 1 \cdot 10^{-3}$ m:

$$\|p_k - p_{k-1}\|_2 > \epsilon$$

Tale verifica evita l'insorgere di singolarità numeriche o segmenti degeneri a lunghezza nulla.

Ottenuto l'insieme filtrato dei punti P , il sistema esegue una parametrizzazione temporale basata sulla lunghezza d'arco. La durata temporale Δt_i allocata per la transizione tra il punto p_i e il punto p_{i+1} è definita dal rapporto con la velocità di crociera nominale V_{cruise} , limitata inferiormente da una costante Δt_{min} s per evitare accelerazioni teoricamente infinite:

$$\Delta t_i = \max \left(\frac{\|p_{i+1} - p_i\|_2}{V_{cruise}}, \Delta t_{min} \right)$$

La variabile temporale globale t è quindi discretizzata in nodi t_i tali che $t_0 = 0$ e $t_i = \sum_{j=0}^{i-1} \Delta t_j$, definendo la durata totale della traiettoria come $t_{end} = t_M$, dove M è il numero di segmenti validati.

6.4.2 Formulazione Analitica delle Spline

La traiettoria è modellata costruendo tre spline cubiche parametriche indipendenti per gli assi cartesiani. Per un generico asse spaziale (ad esempio x), all'interno dell'intervallo temporale $t \in [t_i, t_{i+1}]$, il profilo di posizione è descritto da un polinomio di terzo grado:

$$S_{ix}(t) = a_{ix}(t - t_i)^3 + b_{ix}(t - t_i)^2 + c_{ix}(t - t_i) + d_{ix}$$

L'implementazione utilizza la funzione `CubicSpline` di SciPy. L'algoritmo determina l'insieme dei coefficienti (a, b, c, d) per tutti i segmenti imponendo le seguenti condizioni matematiche di interpolazione e continuità in ogni nodo interno t_i :

1. Passaggio per i punti: $S_{ix}(t_i) = x_i$ e $S_{ix}(t_{i+1}) = x_{i+1}$
2. Continuità di classe C^2 :

$$\dot{S}_{i-1,x}(t_i) = \dot{S}_{ix}(t_i), \quad \ddot{S}_{i-1,x}(t_i) = \ddot{S}_{ix}(t_i)$$

Questa condizione garantisce profili di velocità e accelerazione privi di discontinuità a gradino.

Inoltre, vengono applicate condizioni al contorno di tipo *clamped*, forzando a zero le derivate prime (velocità) agli estremi della traiettoria globale:

$$\dot{S}_x(0) = \dot{S}_y(0) = \dot{S}_z(0) = 0, \quad \dot{S}_x(t_{end}) = \dot{S}_y(t_{end}) = \dot{S}_z(t_{end}) = 0$$

6.5 Sincronizzazione Temporale e Feedforward Cinematico

A differenza delle posizioni cartesiane elaborate analiticamente tramite interpolazione cubica, la gestione dell'orientamento a 6 gradi di libertà è demandata all'algebra dei quaternioni unitari $q = [q_x, q_y, q_z, q_w]^T$.

Già nella fase di pianificazione, il sistema pre-calcola l'assetto ideale per ciascun waypoint:

- Il punto di partenza acquisisce l'orientamento reale e istantaneo letto dall'odometria del drone.
- Il punto di destinazione riceve l'orientamento finale richiesto dalla direttiva spaziale.
- Ai nodi intermedi viene assegnato un orientamento tangente al segmento di volo, calcolato tramite la funzione $\psi = \text{atan2}(\Delta y, \Delta x)$. Assumendo ipotesi di volo planare, la rotazione euleriana viene convertita nel quaternione:

$$q = \begin{bmatrix} 0 \\ 0 \\ \sin(\frac{\psi}{2}) \\ \cos(\frac{\psi}{2}) \end{bmatrix}$$

Una volta ricevuta questa lista di orientamenti formattata, il generatore di traiettoria normalizza i quaternioni per evitare divisioni per zero e imprecisioni numeriche:

$$q_{norm} = \frac{q}{||q||}$$

Successivamente, applica la funzione di Interpolazione Sferica Lineare, o **Slerp**. Lo Slerp garantisce che il drone transiti tra angoli di rotazione differenti descrivendo sempre l'arco minore e a velocità angolare costante nel tempo, evitando strappi sull'asse di imbardata.

Infine, per garantire il tracking preciso in volo, il setpoint cinematico e di assetto inviato al nodo di controllo è così composto:

$$P_{target}(t) = \begin{bmatrix} S_x(t) \\ S_y(t) \\ S_z(t) \end{bmatrix}, \quad V_{target}(t) = \begin{bmatrix} \dot{S}_x(t) \\ \dot{S}_y(t) \\ \dot{S}_z(t) \end{bmatrix}$$

Tale iterazione temporale t non utilizza l'orologio interno del sistema operativo, ma calcola il tempo trascorso avvalendosi rigorosamente dell'orologio di simulazione (`current_sim_time`), estratto dai timestamp dei messaggi `/odom`. Questo garantisce che la valutazione della traiettoria resti indenne ai rallentamenti computazionali o ai cali del Real-Time Factor nel simulatore Gazebo.

Capitolo 7

Navigazione Basata su Human Following

Questo capitolo descrive la progettazione e l'implementazione del modulo di percezione visiva e inseguimento dinamico. Il sistema, integrato nel nodo `yolo_tracker_node`, utilizza una telecamera RGB di bordo per rilevare la presenza di un operatore umano all'interno dell'ambiente logistico, stimarne la posizione tridimensionale e generare automaticamente waypoint di navigazione per il controllore di volo.

7.1 Architettura di Percezione Visiva

Il rilevamento dell'attore dinamico è affidato a una rete neurale convoluzionale di tipo YOLOv8 (versione Nano), ottimizzata per l'inferenza in tempo reale a bordo di veicoli autonomi.

Come evidenziato nell'architettura di sistema, il nodo elabora i fotogrammi estratti dal topic `/camera/image_raw` alla risoluzione di 640x480 pixel.

Per mitigare le oscillazioni del bounding box dovute ai falsi positivi o alle vibrazioni del quadricottero, le coordinate grezze del centroide estratte da YOLO subiscono un filtraggio tramite media mobile esponenziale (EMA):

$$\begin{aligned}u_t &= \alpha u_{raw} + (1 - \alpha)u_{t-1} \\v_t &= \alpha v_{raw} + (1 - \alpha)v_{t-1}\end{aligned}$$

dove $\alpha = 0.5$ rappresenta il fattore di smorzamento. La coordinata orizzontale u_{raw} corrisponde al centro del bounding box, mentre la coordinata verticale v_{raw} viene cam-

pionata sul margine inferiore, in modo da approssimare il punto di contatto tra i piedi dell'operatore e il pavimento del magazzino.



Figura 7.1: Human detection in azione: YOLOv8 identifica il target tra gli scaffali del magazzino.

7.2 Compensazione Odometrica e Ray-Casting

La stima della posizione reale dell'operatore sul piano cartesiano richiede la proiezione del pixel bidimensionale nello spazio 3D. A causa dei ritardi di elaborazione visiva, la posa istantanea del drone al momento dell'arrivo dell'immagine non coincide con quella di acquisizione. Per ovviare a questo problema, il sistema implementa un buffer circolare odometrico (`OdomBuffer`) che interpola linearmente le coordinate spaziali e gli angoli di assetto (Rollio, Beccheggio, Imbardata) per allinearli esattamente al timestamp del fotogramma.

Nota la lunghezza focale f , ricavata dal campo visivo orizzontale (FOV), le coordinate pixel centrate si definiscono come $u_c = u - \frac{W}{2}$ e $v_c = v - \frac{H}{2}$. Il vettore ottico omogeneo viene calcolato e riallineato al frame della telecamera:

$$\vec{r}_{cam} = \begin{bmatrix} 1.0 \\ -u_c/f \\ -v_c/f \end{bmatrix}$$

Successivamente, il raggio viene proiettato nel World Frame applicando la matrice di rotazione del supporto della telecamera R_{mount} (inclinata di 0.349 rad rispetto al telaio) e la matrice di rotazione aerodinamica del drone R_{drone} , ottenuta dalla composizione degli

angoli di Eulero:

$$\vec{r}_{world} = R_{drone} R_{mount} \vec{r}_{cam}$$

Assumendo che il pavimento del magazzino giaccia sul piano $Z = 0$, la distanza di intersezione t lungo la componente verticale D_z del raggio $\vec{r}_{world}$ è calcolata come:

$$t = -\frac{z_{cam}}{D_z}$$

Sostituendo t nell'equazione parametrica della retta, si ottengono le coordinate metriche misurate (x_{meas}, y_{meas}) dell'operatore:

$$x_{meas} = x_{cam} + tD_x$$

$$y_{meas} = y_{cam} + tD_y$$

7.3 Filtraggio tramite Filtro di Kalman

Per garantire un tracciamento persistente anche quando l'operatore viene parzialmente occluso dalle scaffalature industriali, il sistema integra un Filtro di Kalman basato su un modello a velocità costante.

Il vettore di stato è definito dalle posizioni e dalle velocità lineari dell'operatore $\hat{x} = [x, y, v_x, v_y]^T$. All'inizializzazione del filtro, la stima dell'incertezza globale è definita dalla matrice di covarianza dell'errore P_0 , impostata con valori cautelativi molto elevati per riflettere la mancanza di conoscenza a priori sulla posizione esatta:

$$P_0 = \begin{bmatrix} 1000.0 & 0 & 0 & 0 \\ 0 & 1000.0 & 0 & 0 \\ 0 & 0 & 1000.0 & 0 \\ 0 & 0 & 0 & 1000.0 \end{bmatrix}$$

Il processo di predizione aggiorna lo stato basandosi sul tempo intercorso Δt :

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1}$$

$$P_{k|k-1} = FP_{k-1|k-1}F^T + Q$$

A questa fase è associata la matrice di covarianza del rumore di processo Q , che modella l'incertezza intrinseca del modello a velocità costante e le eventuali accelerazioni non previste dell'attore dinamico:

$$Q = \begin{bmatrix} 0.1 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0.1 & 0 \\ 0 & 0 & 0 & 0.1 \end{bmatrix}$$

Infine, la matrice di covarianza del rumore di misura R :

$$R = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

Per rigettare outlier generati da riflessi o falsi positivi di YOLOv8, ogni nuova misurazione z_k è soggetta a un validatore basato sulla Distanza di Mahalanobis:

$$D_M^2 = (z_k - H\hat{x}_{k|k-1})^T S^{-1} (z_k - H\hat{x}_{k|k-1})$$

dove S rappresenta la covarianza dell'innovazione ($S = HP_{k|k-1}H^T + R$). Se la distanza D_M^2 supera la soglia statistica di 9.21 (corrispondente a un intervallo di confidenza del 99

7.4 Logica di Inseguimento e Target Selection

La fase finale del nodo traduce le coordinate filtrate in obiettivi navigabili (Goal) per il pianificatore tridimensionale A^* . Il modulo di Target Selection opera in base a due stati operativi:

- **Fase di RAW Tracking:** Quando la persona è visibile, le sue posizioni metriche vengono salvate in una coda storica. Per prevenire collisioni improvvise dovute a movimenti anomali, l'obiettivo non è l'ultima posizione istantanea, ma il waypoint storico più distante dal drone.
- **Fase di Kalman Tracking:** Se l'operatore esce dal campo visivo (fino a un massimo di 30 frame persi consecutivi), il sistema imposta come obiettivo la posizione (x, y) predetta analiticamente dal Filtro di Kalman.

La logica di attivazione dell'inseguimento si basa su una soglia spaziale (Activation Distance) pari a 5.5 metri. Se la distanza tra il drone e il bersaglio supera tale valore per una durata continua di 1.0 secondo, il sistema innesca il movimento. Tuttavia, per garantire estrema robustezza alle occlusioni visive, questo limite spaziale viene bypassato se il sistema si trova in fase di Kalman Tracking: non appena il target scompare, il drone parte all'inseguimento dell'ultima stima valida a prescindere dalla distanza, cercando di recuperare attivamente la linea di vista.

Superati i controlli di attivazione, il sistema calcola le coordinate del Goal arretrando di 0.4 metri (Stop Distance) lungo la congiungente Drone-Persona, per garantire un margine di frenata. A questo punto, calcola l'angolo di imbardata naturale ($\psi_{traiettoria}$) orientando il vettore verso il punto di arrivo calcolato e pubblica il setpoint forzando l'altitudine operativa a una quota fissa di 3.0 metri. Il raggiungimento della destinazione viene verificato calcolando la distanza euclidea 3D con una tolleranza di 0.2 metri.

Nella pagina successiva è riportato lo pseudocodice dell'algoritmo.

Algorithm 2 Rilevamento e Inseguimento Dinamico (Human Following)

Require: Fotogramma RGB I_t , Odometria sincronizzata P_{drone} , Tempo corrente t

Ensure: Setpoint di navigazione (Goal)

```
1:  $\Delta t \leftarrow t - t_{prev}$ 
2:  $t_{prev} \leftarrow t$ 
3:  $bb_{raw} \leftarrow \text{YOLOv8.Inference}(I_t)$ 
4:  $target_x \leftarrow \text{NULL}$ ,  $target_y \leftarrow \text{NULL}$ 
5:  $tracking\_mode \leftarrow ""$ 

6: FASE 1: Percezione e Filtraggio
7: if  $bb_{raw} \neq \emptyset$  then
8:    $u_{raw}, v_{raw} \leftarrow \text{Calcola\_Centroide\_Inferiore}(bb_{raw})$ 
9:    $u_t, v_t \leftarrow \text{EMA\_Filter}(u_{raw}, v_{raw}, \alpha = 0.5)$ 
10:   $ray_{world} \leftarrow \text{Ray\_Casting}(u_t, v_t, P_{drone})$ 
11:   $x_{meas}, y_{meas} \leftarrow \text{Intersezione\_Piano\_Z0}(ray_{world}, P_{drone})$ 
12:  if  $\text{Mahalanobis\_Distance}(x_{meas}, y_{meas}) \leq 9.21$  then
13:     $\text{KF\_Predict}(\Delta t)$ 
14:     $\text{KF\_Update}(x_{meas}, y_{meas})$ 
15:     $\text{Path\_History.append}(x_{meas}, y_{meas})$ 
16:     $lost\_frames \leftarrow 0$ 
17:     $target_x, target_y \leftarrow \text{Get\_Farthest\_Point}(\text{Path\_History}, P_{drone})$ 
18:     $tracking\_mode \leftarrow "RAW"$ 
19:  end if
20: else
21:   $\text{KF\_Predict}(\Delta t)$ 
22:   $lost\_frames \leftarrow lost\_frames + 1$ 
23:  if  $lost\_frames < 50$  then
24:     $target_x, target_y \leftarrow \text{KF\_Get\_State\_Position}()$ 
25:     $tracking\_mode \leftarrow "KALMAN"$ 
26:  end if
27: end if

28: FASE 2: Logica di Attivazione e Navigazione
29: if  $target_x \neq \text{NULL}$  and  $target_y \neq \text{NULL}$  and not  $is\_moving$  then
30:   $dist \leftarrow \text{Euclidean\_Distance}(P_{drone}.x, P_{drone}.y, target_x, target_y)$ 
31:  if  $dist \geq 5.5$  or  $tracking\_mode == "KALMAN"$  then
32:    if  $Activation\_Time == \text{NULL}$  then
33:       $Activation\_Time \leftarrow t$ 
34:    else if  $(t - Activation\_Time) \geq 1.0$  then
35:       $angle\_to\_target \leftarrow \text{atan2}(target_y - P_{drone}.y, target_x - P_{drone}.x)$ 
36:       $goal_x \leftarrow target_x - 0.6 \cdot \cos(angle\_to\_target)$ 
37:       $goal_y \leftarrow target_y - 0.6 \cdot \sin(angle\_to\_target)$ 
38:       $goal_z \leftarrow 3.0$ 
39:       $trajectory\_yaw \leftarrow \text{atan2}(goal_y - P_{drone}.y, goal_x - P_{drone}.x)$ 
40:       $\text{Invia\_Goal\_AlPianificatore}(goal_x, goal_y, goal_z, trajectory\_yaw)$ 
41:       $is\_moving \leftarrow \text{TRUE}$ 
42:       $Activation\_Time \leftarrow \text{NULL}$ 
43:    end if
44:  else
45:     $Activation\_Time \leftarrow \text{NULL}$ 
46:  end if
47: end if
```

Capitolo 8

Simulazione e Validazione Sperimentale

L'obiettivo di questo capitolo è validare empiricamente l'architettura software proposta, dimostrando l'efficacia degli algoritmi di percezione, pianificazione e controllo implementati in ambiente ROS 2 e simulati all'interno di Gazebo.

Al fine di valutare in modo completo e organico le prestazioni del sistema, la campagna di test è stata suddivisa in due macro-scenari operativi. La prima sezione è dedicata alla valutazione della navigazione spaziale verso punti discreti in presenza di ostacoli statici, mentre la seconda analizza le capacità di reazione e inseguimento dinamico di un bersaglio in movimento

8.1 Decollo e Stabilizzazione in Hovering

Prima di procedere con l'analisi delle traiettorie di navigazione complesse, è fondamentale validare la stabilità dell'architettura di controllo in cascata illustrata precedentemente.

A tal fine, il primo test simulato ha lo scopo di valutare la reazione del quadricottero partendo da uno stato di quiete a terra, imponendo un setpoint statico iniziale.

Il nodo di controllo è stato configurato per raggiungere e mantenere una quota operativa di **3.0 m**, eseguendo contestualmente una rotazione sull'asse di imbardata per portarsi a un angolo di **90°**. Le coordinate planari e gli angoli di assetto (rollio e beccheggio) sono rigorosamente vincolati a un setpoint nullo.

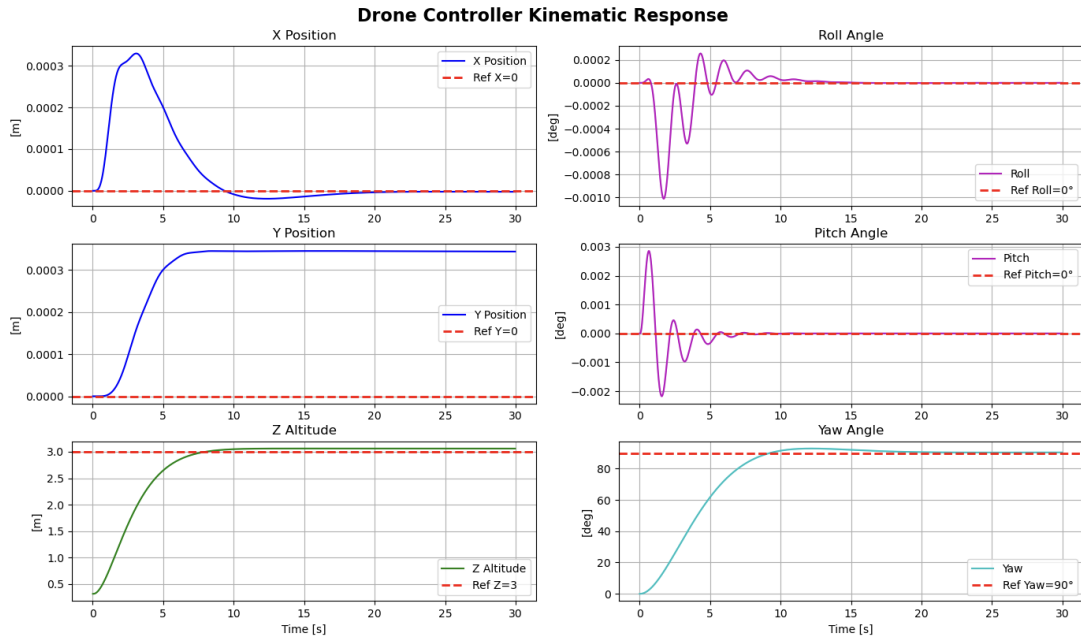


Figura 8.1: Risposta cinematica del drone

Il drone raggiunge la quota target di 3 m e si allinea al riferimento di 90° in circa 10 secondi. Le curve si presentano estremamente fluide e asintotiche, prive di sovraelongazioni. Come si nota dalle scale dei grafici, gli scostamenti massimi sono nell'ordine dei decimi di millimetro per le posizioni X e Y e di millesimi di grado per rollio e beccheggio.

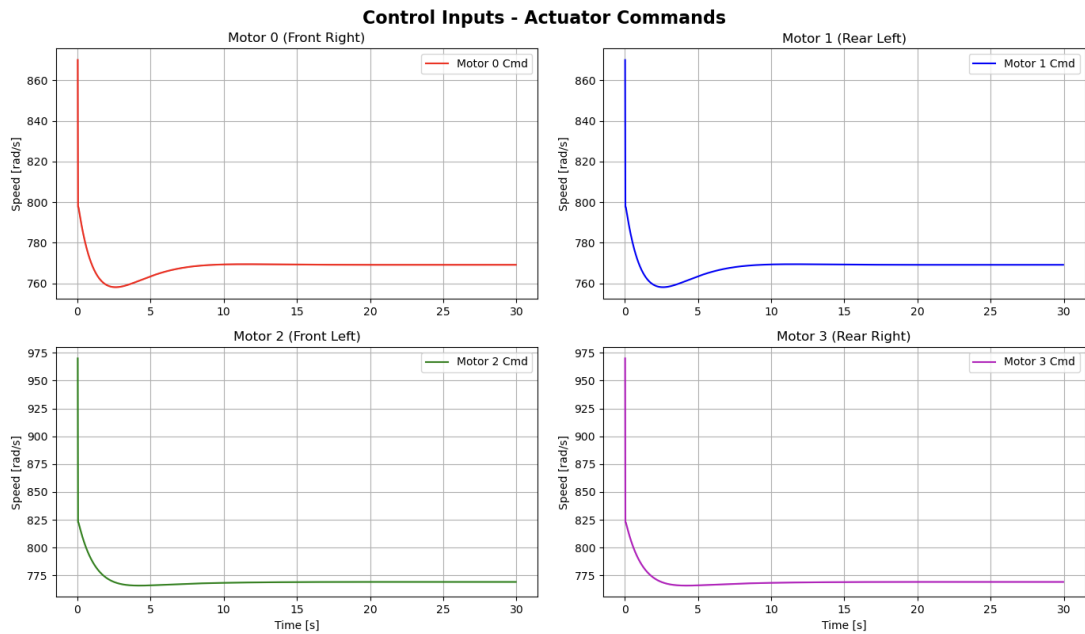


Figura 8.2: Comandi di velocità angolare degli attuatori

Nei primi istanti della simulazione si osserva un picco nelle velocità angolari, necessario per generare una forza di spinta superiore alla forza peso.

Emerge un netto differenziale tra i rotori: i motori 2 e 3 (rotazione oraria) raggiungono un picco transitorio di circa 970 rad/s, mentre i motori 0 e 1 (rotazione antioraria) si

limitano a 870 rad/s. Questo sbilanciamento asimmetrico è indotto volontariamente dal PID di imbardata per creare il momento torcente netto richiesto dalla virata iniziale. Raggiunta la quota target e completata la rotazione, l'azione derivativa e proporzionale dei regolatori si esaurisce. I comandi dei quattro motori convergono dinamicamente verso un valore stazionario comune di circa 770 rad/s (`base_thrust`). Infine, durante l'intero arco temporale del test, le uscite elaborate dal sistema rispettano rigorosamente la funzione di saturazione geometrica, mantenendosi costantemente all'interno del limite fisico tollerato dai rotori, impostato a 1000 rad/s.



Figura 8.3: Drone in hovering all'interno dell'ambiente di simulazione

8.2 Navigazione 3D Autonoma

Questa fase è stata progettata per validare l'integrazione tra il modulo di percezione spaziale, il pianificatore di traiettoria e l'anello di controllo. I test sono stati condotti assegnando dinamicamente coordinate di destinazione nello spazio tridimensionale, imponendo al velivolo di calcolare ed eseguire percorsi complessi che includono variazioni simultanee di posizione planare e altitudine.

Per innescare la generazione del percorso e il successivo volo, i setpoint di destinazione sono stati inviati al nodo di pianificazione in modo asincrono sfruttando l'interfaccia a riga di comando di ROS 2.

8.2.1 Scenario A: Navigazione Planare a Quota Costante

Nel primo scenario, al drone è stato richiesto di raggiungere un obiettivo variando unicamente le proprie coordinate planari (X, Y), mantenendo inalterata l'altitudine opera-

tiva. Questo test è cruciale per verificare l'isolamento tra l'anello di controllo esterno (traslazione orizzontale) e l'anello interno (altitudine e assetto).

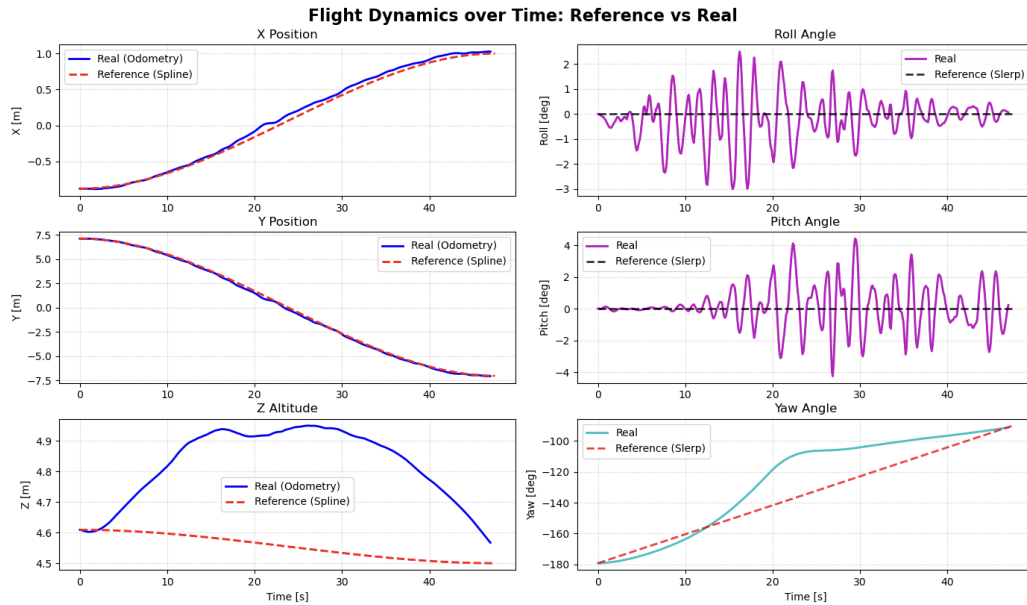


Figura 8.4: Risposta cinematica durante una navigazione prolungata a quota costante

Come si evince dai grafici temporali, il sistema reagisce in modo robusto, evidenziando le seguenti dinamiche:

- **Inseguimento Orizzontale (X, Y):** Le curve relative agli assi planari dimostrano che l'odometria reale (linea blu continua) si sovrappone in modo eccellente ai profili spaziali di riferimento (linea tratteggiata rossa). L'assenza di un errore transitorio marcato certifica il successo dell'architettura predittiva implementata: il guadagno di feedforward iniettato nell'anello esterno anticipa le richieste di spostamento, garantendo un tracking fluido lungo l'intera traiettoria.
- **Gestione dell'Assetto (Roll, Pitch, Yaw):** Per garantire l'avanzamento planare, l'anello esterno genera in tempo reale i setpoint di inclinazione. I subplot di rollio e beccheggio mostrano un comportamento oscillatorio fisiologico, necessario per micro-correggere la rotta, ma rigorosamente confinato in un range di sicurezza molto ristretto (tra -4° e $+4^\circ$). L'angolo di imbardata (Yaw) insegue il riferimento con una dinamica dolce, completando una rotazione da -180° a circa -90° per allineare costantemente il muso del velivolo alla direzione del vettore velocità.
- **Mantenimento della Quota (Z) e Cross-Coupling:** Il profilo di altitudine rappresenta il dato fisicamente più rilevante del test. A fronte di un riferimento quasi piatto, l'odometria reale registra un innalzamento transitorio prolungato, raggiungendo un picco di scostamento massimo di circa 0.35 m (da 4.60 m a 4.95 m).

Questo fenomeno non rappresenta un'instabilità, bensì un noto effetto aerodinamico definito *cross-coupling* (accoppiamento trasversale): l'inclinazione prolungata del telaio per mantenere la velocità di crociera in X-Y altera la scomposizione vettoriale della spinta verticale. Il regolatore PID dell'altitudine assorbe questa perturbazione in modo smorzato, impedendo divergenze pericolose e riportando gradualmente la quota verso il setpoint ideale durante la fase di decelerazione finale.

8.2.2 Scenario B: Navigazione Tridimensionale a Quota Variabile

Nel secondo scenario, il drone è stato sottoposto a uno stress test di navigazione ad elevata complessità spaziale. Nello specifico, il quadricottero si trovava in hovering dietro una scaffalatura di grandi dimensioni e l'obiettivo è stato posizionato esattamente sul lato opposto dell'ostacolo. Questa configurazione ha costretto l'algoritmo A^* tridimensionale a pianificare una manovra di "scavalco", richiedendo un forte dislivello altimetrico combinato a uno spostamento simultaneo sul piano orizzontale.

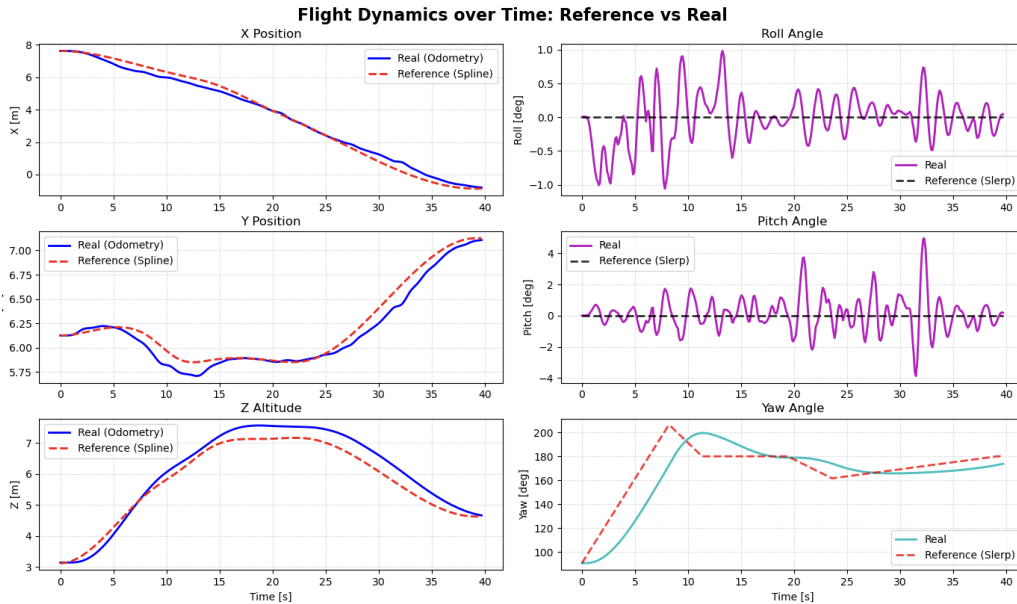


Figura 8.5: Risposta cinematica durante il superamento di un ostacolo strutturale.

Analizzando le dinamiche temporali per questo scenario, emergono prestazioni eccellenti nella gestione simultanea dei tre gradi di libertà traslazionali:

- **Gestione dell'Ostacolo e Altitudine (Z):** Per evitare la collisione, la spline di riferimento genera una ripida rampa di salita, portando la quota dai 3.0 m iniziali fino a un picco di oltre 7.0 m, per poi ridiscendere dolcemente verso i 4.5 m finali. L'odometria reale insegue questa complessa traiettoria a campana con notevole

precisione. Si osserva un lievissimo ritardo di fase durante la salita e un inerziale ma contenuto overshoot al vertice (circa 0.2 m rispetto al riferimento); un comportamento del tutto fisiologico per un multirobotore impegnato a saturare i motori per vincere la gravità mentre accelera orizzontalmente.

- **Inseguimento Planare Complesso (X, Y):** Nonostante il forte impegno del sistema propulsivo per la trazione verticale, il tracking planare non subisce degradi. L'asse X copre un'escursione di quasi 9 m in modo asintotico e continuo, mentre l'asse Y segue un profilo spaziale non lineare, calcolato dinamicamente dal pianificatore. L'azione del feedforward cinematico garantisce che l'odometria si mantenga saldamente incollata al riferimento per tutta la durata del volo (circa 40 secondi).
- **Risposta di Assetto (Roll, Pitch) e Imbardata (Yaw):** Le sollecitazioni di rollio risultano minime e contenute entro una ristretta banda di $\pm 1^\circ$, mentre il beccheggio lavora in un range di sicurezza compreso tra -4° e $+5^\circ$ per assecondare le repentine accelerazioni frontali. Risulta di particolare interesse ingegneristico il grafico dell'imbardata: il riferimento geometrico (Slerp) presenta un repentino cambio di pendenza attorno al decimo secondo, dovuto al ricalcolo della tangente sulla curva 3D. Il PID di imbardata reale agisce come un filtro naturale, addolcendo questa discontinuità ed eseguendo la rotazione direzionale in modo progressivo, evitando di trasmettere pericolosi scatti meccanici al telaio del drone.

Ulteriori validazioni tridimensionali

A ulteriore conferma della robustezza e della ripetibilità del sistema, la Figura 8.6 mostra una diversa traiettoria tridimensionale. Come nei casi precedenti, l'algoritmo garantisce un inseguimento spaziale preciso, connettendo fluidamente i waypoint e mantenendo un allineamento coerente dell'imbardata lungo l'intera curva.

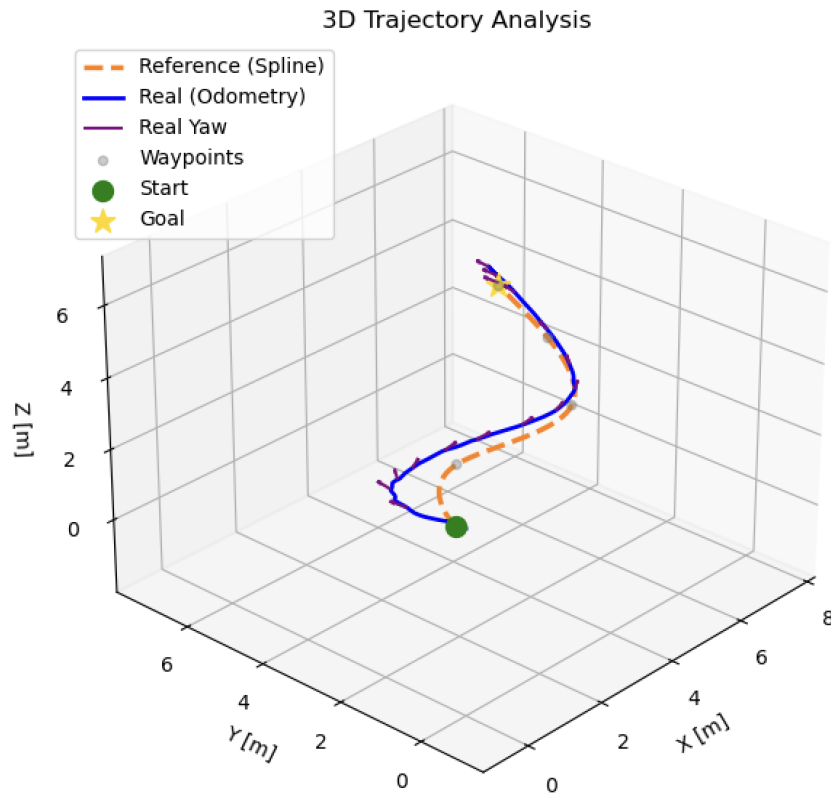


Figura 8.6: Esempio di traiettoria 3D complessa.

8.2.3 Analisi del Grafo Computazionale ROS 2

A conclusione dell'analisi della navigazione tridimensionale, si riporta la topologia della rete computazionale attiva durante la simulazione, estratta mediante lo strumento `rqt`.

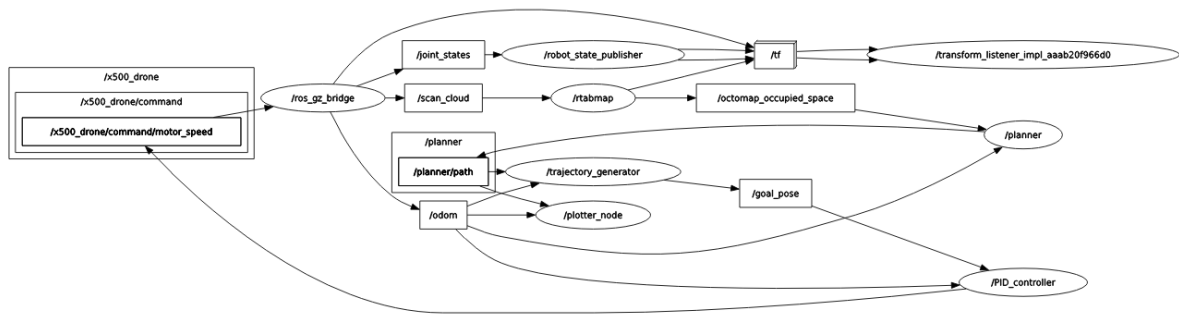


Figura 8.7: Grafo computazionale durante l'esecuzione della navigazione autonoma.

L'analisi del grafo illustra il flusso deterministico delle informazioni e la gerarchia dei sottosistemi:

- **Interfaccia di Simulazione:** Il nodo `/ros_gz_bridge` funge da ponte bidirezionale tra il motore fisico Gazebo e l'ecosistema ROS 2. Esso pubblica in tempo reale i dati odometrici (`/odom`), le letture del LiDAR (`/scan_cloud`) e lo stato dei giunti (`/joint_states`).
- **Mappatura Spaziale:** Il nodo `/rtabmap` elabora la nuvola di punti grezza ricevuta dal ponte e la integra con le trasformazioni spaziali (`/tf`), pubblicando la mappa tridimensionale discretizzata sul topic `/octomap_occupied_space`.
- **Pianificazione e Traiettorie:** La scomposizione del problema di navigazione discussa nel Capitolo 6 è qui chiaramente visibile. Il nodo `/planner` interroga la mappa volumetrica e pubblica i waypoint discreti su `/planner/path`. Questo topic viene intercettato dal nodo `/trajectory_generator`, il quale interpola i punti per generare la traiettoria spline parametrizzata, inviando i setpoint cinematici continui sul topic `/goal_pose`.
- **Controllo di Volo e Attuazione:** Il nodo `/PID_controller` chiude l'anello di controllo. Iscrivendosi sia all'odometria reale che al target dinamico, calcola l'azione di controllo e trasmette le velocità angolari dei rotori sul topic `/x500_drone/command/motor_speed`, che il bridge instrada nuovamente verso il modello fisico in Gazebo.
- **Logging Visivo:** Infine, la presenza del nodo ausiliario `/plotter_node`, iscritto all'odometria e al percorso pianificato, giustifica la generazione automatica dei report grafici analizzati nei paragrafi precedenti.

Nel grafo è visibile un nodo generato dinamicamente con nomenclatura `/transform_listener_impl_...`. Si tratta di un processo ausiliario istanziato in automatico dalle librerie core di ROS 2 (nello specifico `tf2_ros`). Questo listener opera in background e si iscrive al topic `/tf` per mantenere un buffer asincrono e *thread-safe* dell'albero delle trasformazioni spaziali (come le relazioni tra i frame `map`, `odom`, `base_link` e il sensore LiDAR). La sua presenza è indispensabile per garantire la rigorosa coerenza geometrica delle operazioni svolte dal pianificatore e dal modulo di SLAM senza bloccare il flusso di esecuzione principale.

8.3 Navigazione con Humang Following

L'ultima fase della è dedicata alla validazione del sistema di inseguimento dinamico descritto nel Capitolo 7. A differenza dei test precedenti, in questo scenario il drone non riceve coordinate spaziali statiche da un operatore esterno, ma chiude un anello di percezione-azione generando autonomamente i propri waypoint di navigazione, inseguendo un attore in movimento all'interno del magazzino.



Figura 8.8: Drone in fase di inseguimento autonomo nell'ambiente di simulazione

8.3.1 Analisi Spaziale della Traiettoria di Inseguimento

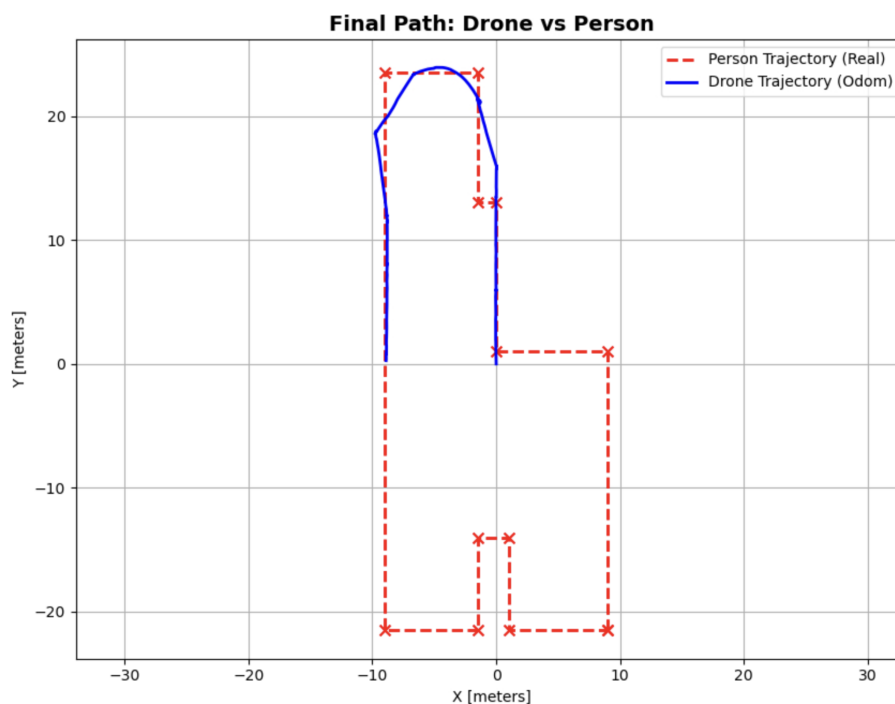


Figura 8.9: Mappatura 2D (X-Y) del percorso (Simulazione 1).

Analizzando il tracciato in Figura (8.9) e (8.10)(relativo a un pezzo di simulazione interrotta manualmente), si evince:

- **Smussamento Dinamico:** A differenza del target che esegue virate secche a 90° , il drone raccorda i cambi di direzione con curve fluide. Ciò dimostra l'efficacia dell'interpolazione Cubic Spline, necessaria per mantenere un moto continuo e rispettare i vincoli aerodinamici del velivolo.
- **Recupero dell'Inerzia:** Durante le inversioni a "U", il drone allarga fisiologicamente la traiettoria per dissipare l'inerzia, per poi riallinearsi rapidamente al bersaglio grazie al ricalcolo continuo del pianificatore spaziale.
- **Robustezza alle Occlusioni Visive:** Nei tratti in cui l'operatore svolta dietro le scaffalature, la linea di vista si spezza e YOLO smette di rilevare il bersaglio. Il grafico dimostra come il drone non si arresti passivamente, ma sfrutti con successo la predizione analitica dello stato fornita dal Filtro di Kalman per proseguire l'inseguimento "alla cieca", aggirando l'ostacolo strutturale e recuperando in autonomia il contatto visivo.

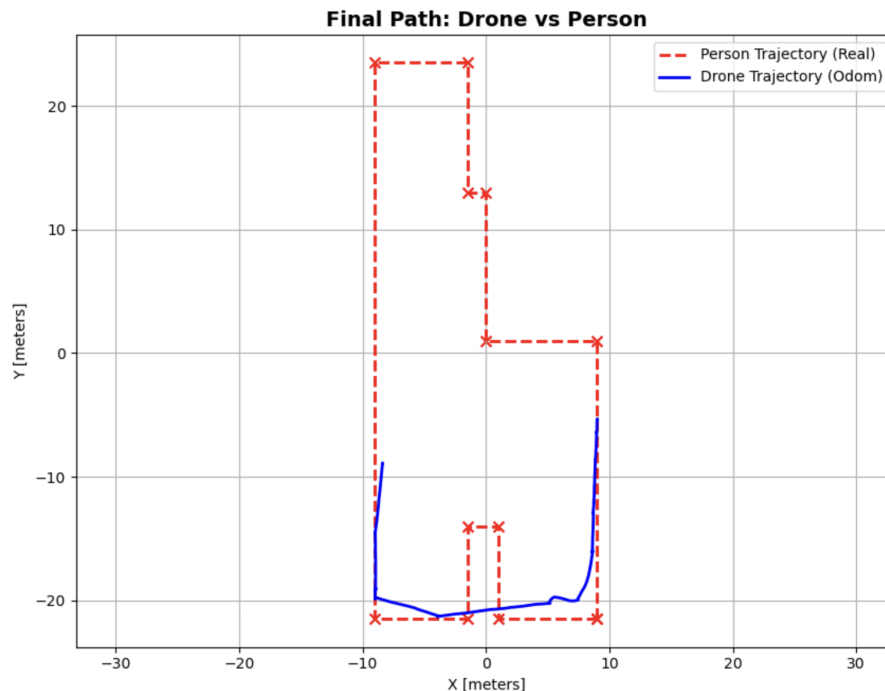


Figura 8.10: Mappatura 2D (X-Y) del percorso (Simulazione 2).

8.3.2 Analisi del Grafo Computazionale ROS 2

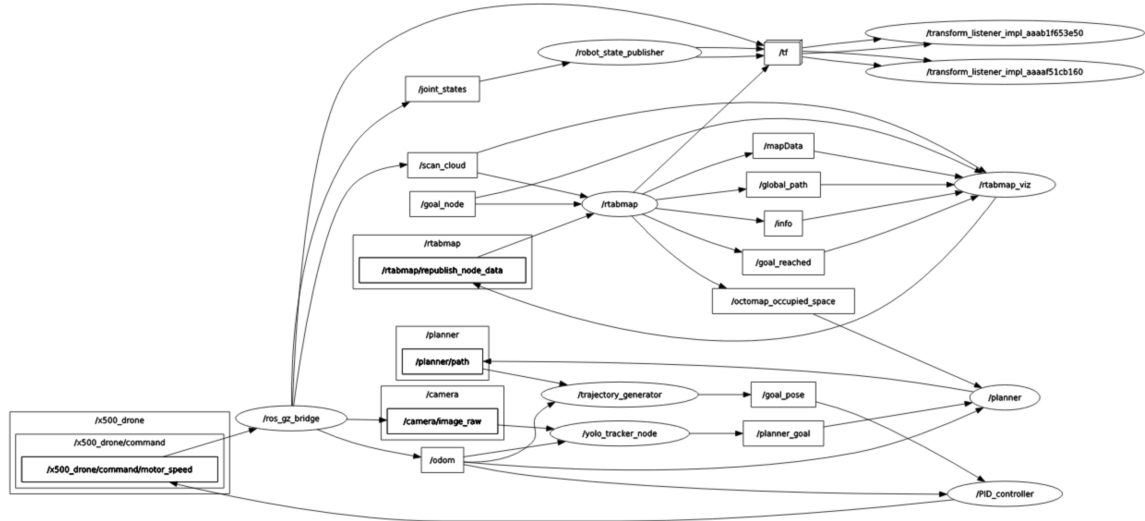


Figura 8.11: Grafo computazionale durante l'esecuzione del task di human following.

In questo scenario, il tracker va a sostituire completamente l'intervento dell'operatore umano (che nei test precedenti forniva i waypoint in modo asincrono tramite interfaccia a riga di comando).

Come visibile dal grafo, il nuovo modulo si innesta nell'architettura prelevando il flusso video RGB (`/camera/image_raw`) e lo stato cinematico del drone (`/odom`), entrambi forniti dal bridge di Gazebo.

A valle del riconoscimento visivo, della compensazione geometrica e del filtraggio di Kalman, il nodo pubblica i setpoint dinamici sul topic `/planner_goal`.